

NANOQUANT: Efficient Sub-1-Bit Quantization of Large Language Models

Hyochan Chong^{*1} Dongkyu Kim^{*†1} Changdong Kim¹ Minseop Choi¹

Abstract

Weight-only quantization has become a standard approach for efficiently serving large language models (LLMs). However, existing methods fail to efficiently compress models to binary (1-bit) levels, as they either require large amounts of data and compute or incur additional storage. In this work, we propose NANOQUANT, a post-training quantization (PTQ) method to compress LLMs to both binary and *sub-1-bit* levels. NANOQUANT formulates quantization as a low-rank binary factorization problem, and compresses full-precision weights to low-rank binary matrices and scales. Specifically, it utilizes an efficient alternating direction method of multipliers (ADMM) solver to precisely initialize latent binary matrices and scales, and then tunes the initialized parameters through a block and model reconstruction process. Consequently, NANOQUANT establishes a new Pareto frontier in low-memory post-training quantization, and enables sub-1-bit compression. NANOQUANT makes large-scale deployment feasible on consumer hardware. For example, it compresses Llama-2-70B by 24× in just 13 hours on a *single* H100, enabling a 70B model to operate on a consumer 8 GB GPU. Code is available at github.com/SamsungLabs/NanoQuant.

1. Introduction

Large language models (LLMs) have demonstrated remarkable performance across a wide variety of tasks. However, their extremely large size makes deployment costly. Weight-only quantization offers a standard route to alleviate these bottlenecks (Frantar et al., 2022; Lin et al., 2024; Shao et al., 2024; Liu et al., 2025). This has led to its widespread adoption within production-grade inference engines, such as vLLM (Kwon et al., 2023) and SGLang (Zheng et al.,

Table 1. Comparison of LLM quantization frameworks. Methods are categorized by quantization scheme (PTQ vs. QAT), scalability to 70B+ models, and sub-1-bit capability. Only NANOQUANT enables sub-1-bit compression among baselines.

Quantization Method	Scheme	Compression		
		70B+ LLMs	1-Bit	Sub-1-Bit
BiLLM (Huang et al., 2024)	PTQ	✓	✗	✗
STBLLM (Dong et al., 2025)	PTQ	✓	✗	✗
ARB-LLM (Li et al., 2025)	PTQ	✓	✗	✗
HB-LLM (Chen et al., 2026)	PTQ	✓	✗	✗
OneBit (Xu et al., 2024)	QAT	✗	✓	✗
BinaryMoS (Jo et al., 2024)	QAT	✗	✓	✗
DBF (Boža & Macko, 2026)	QAT	✗	✓	✗
ParetoQ (Liu et al., 2026)	QAT	✗	✓	✗
LittleBit (Lee et al., 2025a)	QAT	✗	✓	✓
NANOQUANT (Ours)	PTQ	✓	✓	✓

2024).

Recent post-training quantization (PTQ) efforts have successfully pushed weight compression toward 2-bit (Chee et al., 2023; Tseng et al., 2024b) and even 1-bit (Huang et al., 2024; Li et al., 2025; Chen et al., 2026). However, breaking the *sub-1-bit* barrier remains a challenge for current PTQ frameworks for two distinct reasons. First, current binary PTQ methods utilize in-place binarization with full-precision scales (*e.g.*, $\mathbf{W} \approx \alpha \mathbf{B}_{\pm 1}$), an approach that is structurally bounded by a minimum of 1 bit per parameter. Moreover, these techniques require complex weight-grouping metadata (Huang et al., 2024; Dong et al., 2025; Zhao et al., 2025; Chen et al., 2026), causing effective bit-rates to exceed 2 and even 3 bits (Dong et al., 2025). Thus, a key challenge for sub-1-bit PTQ is to efficiently represent model parameters to overcome both the structural and storage limitations of current methods.

In contrast, binary quantization-aware training (QAT) methods successfully compress LLMs to binary (1-bit) and even sub-1-bit levels with low-rank representations (Lee et al., 2025a; Boža & Macko, 2026), overcoming both the structural and additional storage limitations inherent in binary PTQ methods. Through an end-to-end training process, such binary QAT methods replace linear layer weights with compact, low-rank binary matrices and scales. However, unlike PTQ methods, these QAT methods require hundreds of millions or billions of tokens, and utilize multiple GPUs over multiple days. These demands are impractical for resource-constrained environments and limit such QAT methods from

^{*}Equal contribution ¹Samsung Research, Seoul, Korea.

[†]Correspondence to: Dongkyu Kim <dongkyu.k@samsung.com>.

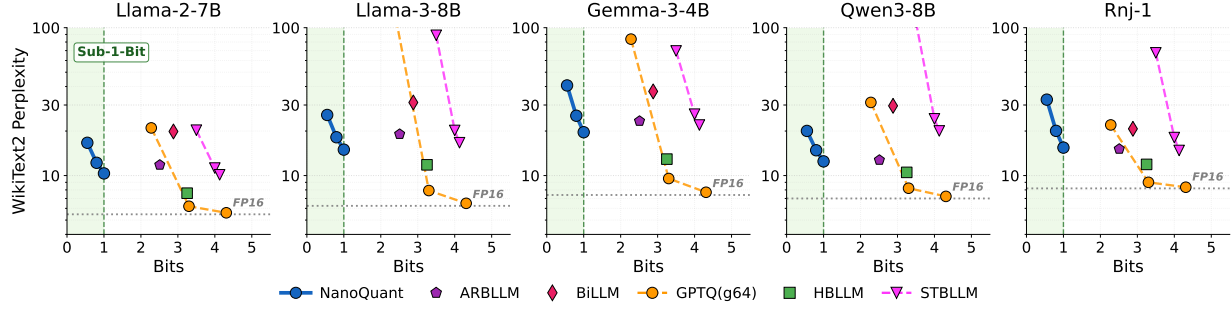


Figure 1. Perplexity comparison on WikiText-2. NANOQUANT reaches 1-bit and sub-1-bit effective storage while remaining competitive with existing binary PTQ baselines that require substantially higher effective BPW.

compressing larger, 70B parameter models. Therefore, deriving a *data-efficient* and *compute-efficient* sub-1-bit PTQ method remains an open challenge.

To bridge the gap between binary PTQ and QAT methods, we propose NANOQUANT, an efficient and accurate PTQ method that can compress LLMs to *sub-1-bit* levels. By directly addressing multiple shortcomings of conventional post-training quantization methods, NANOQUANT precisely initializes latent binary matrices and scales via robust Hessian-aware alternating direction method of multipliers (ADMM). Then, NANOQUANT utilizes a hierarchical reconstruction pipeline that optimizes parameters at the block level, and subsequently calibrates scaling factors at the model level for enhanced global activation alignment. With only 128 calibration samples (0.26M tokens) and 1 GPU, NANOQUANT achieves sub-binary compression and shows competitive performance in low-memory regimes. NANOQUANT enables compressing a 70B LLM from 137.95 GB to 5.75 GB using 1 GPU, and running the quantized 70B LLM on a consumer 8 GB GPU at up to 20.11 tokens per second, making LLM compression and inference accessible in resource-constrained environments.

Contributions. Our main contributions are as follows:

- We propose NANOQUANT, a post-training quantization (PTQ) method to compress LLMs to both 1-bit and sub-1-bit levels. This approach addresses the structural limitations of existing binary quantization frameworks.
- We provide a stability analysis of the initialization procedure and empirically show that precise low-rank binary initialization is critical for establishing a new sub-1-bit quantization frontier.
- We conduct extensive experiments across diverse model families and language tasks, demonstrating that NANOQUANT achieves competitive performance with higher-bit PTQ and binary quantization-aware training (QAT) methods, despite using limited calibration data.

- We implement custom binary GEMV and GEMM CUDA kernels for NANOQUANT, enabling significantly higher inference throughput, reduced memory footprints, and enhanced energy efficiency for datacenter GPUs, consumer GPUs, and edge devices.

2. Related Work

Binary Post-Training Quantization. State-of-the-art binary post-training quantization (PTQ) methods often adopt in-place binarization and full-precision scales to preserve the sign and magnitude of weights, respectively (Huang et al., 2024; Li et al., 2025; Chen et al., 2026). Other methods introduce sparsity to further reduce the memory footprint of binary weights (Dong et al., 2025). However, although such methods show respectable performance, these binary PTQ algorithms incur additional storage requirements (e.g. scaling factors and grouping bit-masks), causing them to fall short of their intended binary compression rates, requiring at least 2 or 3 bits per weight (Huang et al., 2024; Li et al., 2025; Zhao et al., 2025; Chen et al., 2026). QMoE (Frantar & Alistarh, 2024) and BTC-LLM (Gu et al., 2025) are notable methods that achieve sub-1-bit levels, but they respectively target mixture-of-experts models or utilize codebooks with additional storage overhead.

Binary Quantization-Aware Training. In contrast, binary quantization-aware training (QAT) methods successfully reach binary compression rates through end-to-end training on larger datasets. Many previous binary QAT methods utilize in-place binarization to compress LLM weights to binary levels (Wang et al., 2023; Xu et al., 2024; Jo et al., 2024; Liu et al., 2026). More recent methods compress weights to low-rank binary matrices to reach binary and sub-1-bit compression levels (Lee et al., 2025a; Boža & Macko, 2026). However, although such low-rank binary methods display promising performance, they require copious amounts of data and compute, requiring multiple GPUs for multiple days to train on hundreds of millions or billions of tokens. Such resource demands have also limited these

methods to binarize only relatively smaller models, such as Llama-2-7B.

Alternating Direction Method of Multipliers. Alternating Direction Method of Multipliers (ADMM) is a classical framework for constrained optimization that alternates between augmented-Lagrangian subproblems and dual-variable updates (Glowinski & Marroco, 1975; Gabay & Mercier, 1976; Boyd et al., 2011). Although ADMM is best understood in convex optimization (Boyd & Vandenberghe, 2004; Bertsekas, 2014), it has also been studied in nonconvex and nonsmooth settings (Hong et al., 2016; Wang et al., 2019; Yang et al., 2022; Barber & Sidky, 2024). This makes ADMM a natural fit for model compression problems that combine continuous reconstruction objectives with hard structural constraints. Prior work has applied ADMM to low-bit quantization (Leng et al., 2018; Xu et al., 2020; 2021; Huang et al., 2021; Boža & Macko, 2026), quantization-aware factorization (Cherniuk et al., 2024), sparsification (Holmes et al., 2021; Meng et al., 2024; Lee et al., 2025b), and sparse-plus-low-rank LLM compression (Makni et al., 2025). In NANOQUANT, we use ADMM in this spirit to decouple weight reconstruction from the discrete low-rank binary structure, enabling efficient optimization of sub-1-bit LLM weight decompositions.

3. NANOQUANT

This section introduces NANOQUANT, a post-training quantization (PTQ) method capable of compressing LLM weights to sub-1-bit levels. NANOQUANT derives high-fidelity low-rank binary representations by integrating a precise initialization subroutine directly into a block-wise reconstruction loop, followed by lightweight global calibration.

3.1. Quantization Scheme

We formulate sub-1-bit weight compression as a low-rank binary factorization problem, similar to (Lee et al., 2025a; Boža & Macko, 2026). Let $\mathbb{B} = \{-1, +1\}$ denote the set of binary values. For each linear layer weight $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ in the transformer, we approximate the dense matrix using two low-rank binary matrices $\mathbf{U}_{\pm 1} \in \mathbb{B}^{d_{\text{out}} \times r}$ and $\mathbf{V}_{\pm 1} \in \mathbb{B}^{d_{\text{in}} \times r}$, alongside two full-precision scaling vectors: an output channel scale $\mathbf{s}_1 \in \mathbb{R}^{d_{\text{out}}}$ and an input channel scale $\mathbf{s}_2 \in \mathbb{R}^{d_{\text{in}}}$.

The decomposition structure is defined as

$$\mathbf{W} \approx \widehat{\mathbf{W}} = \mathbf{s}_1 \odot (\mathbf{U}_{\pm 1} \mathbf{V}_{\pm 1}^T) \odot \mathbf{s}_2^T, \quad (1)$$

where $\odot$ denotes element-wise multiplication with broadcasting. Figure 2 visualizes this scheme, illustrating how the dense weight matrix decomposes into continuous latent factors and scales before binarization and packing. Di-

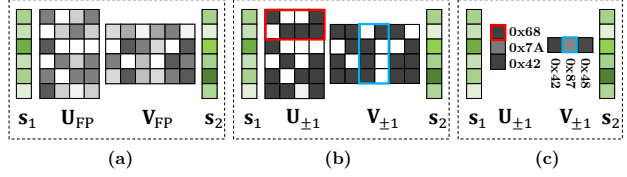


Figure 2. Illustration of the NANOQUANT compression scheme. The process proceeds in three stages: (a) **Factorization**, where the weight matrix is decomposed into continuous latent factors ($\mathbf{U}_{\text{FP}}, \mathbf{V}_{\text{FP}}$) and floating-point scales ($\mathbf{s}_1, \mathbf{s}_2$) which are fine-tuned to minimize reconstruction error; (b) **Binarization**, where these optimized factors are quantized into binary matrices ($\mathbf{U}_{\pm 1}, \mathbf{V}_{\pm 1}$) containing $\{-1, +1\}$ values; and (c) **Packing**, where these values are mapped to bits ($-1 \rightarrow 0, +1 \rightarrow 1$) and efficiently packed into integer formats (e.g., 8-bit blocks) for memory efficiency.

rect optimization of binary parameters constitutes a non-convex, combinatorial problem that is NP-hard (Froese & Hertrich, 2023). To address this within a strict PTQ budget, NANOQUANT employs a sequential block reconstruction pipeline that incorporates precise initialization and latent optimization.

3.2. Block Reconstruction Pipeline

We sequentially compress each linear layer in each transformer block. Unlike methods that treat initialization as a separate pre-processing phase, NANOQUANT integrates initialization as a subroutine within the block reconstruction loop. As depicted in Figure 3, each block undergoes a three-step optimization process: (1) error propagation mitigation, (2) low-rank binary initialization via ADMM and magnitude balancing, and (3) factorized component refinement.

Step 1: Error Propagation Mitigation. Quantization error accumulates as the reconstruction proceeds through the network (Frantar et al., 2022). We tune the full-precision weights of the current block to minimize the error introduced by the quantization of preceding blocks, as well as previously factorized layers in the current block. This comprehensive strategy is in line with recent quantization methods that adopt this method for either some (Boža & Macko, 2026) or all (Tseng et al., 2024a; Egiazarian et al., 2024; Arai & Ichikawa, 2026) linear layers, and NANOQUANT falls in the latter.

Step 2: Low-Rank Binary Initialization. Because PTQ relies on a small calibration set, the stability of initialization is critical (Hubara et al., 2021; Nagel et al., 2020). We initialize the low-rank binary parameters and scales through an activation-aware process involving preconditioning, factorization via alternating direction method of multipliers (ADMM), and magnitude balancing.

Step 2-1: Hessian-Aware Preconditioning. To minimize quantization error, we adopt the formulation from DBF (Boža & Macko, 2026) and consider the second-order Taylor expansion of the task loss. The objective minimizes the Hessian-weighted distortion approximated via Kronecker-factored approximate curvature (K-FAC) (Martens & Grosse, 2015):

$$\mathcal{L}(\widehat{\mathbf{W}}) \approx \|\widetilde{\mathbf{D}}_{\text{out}}(\mathbf{W} - \widehat{\mathbf{W}})\widetilde{\mathbf{D}}_{\text{in}}\|_F^2. \quad (2)$$

Here, $\widetilde{\mathbf{D}}_{\text{in}}$ and $\widetilde{\mathbf{D}}_{\text{out}}$ are diagonal preconditioners constructed from activation and gradient statistics. These values are computed during a global calibration phase prior to the block-wise reconstruction loop, as outlined in Algorithm 1. Given limited calibration data, empirical estimates can be sensitive to outliers. To mitigate this, we employ shrinkage (Ledoit & Wolf, 2004) regularization on the diagonal entries,

$$[\widetilde{\mathbf{D}}(\cdot)]_{ii} \leftarrow (1 - \gamma)[\mathbf{D}(\cdot)]_{ii} + \gamma \text{mean}(\mathbf{D}(\cdot)). \quad (3)$$

The shrinkage coefficient $\gamma \in [0, 1]$ plays a pivotal role in regulating the trade-off between preserving feature-specific curvature information and maintaining global robustness against calibration noise. We empirically find that smaller values (e.g., 0.2) are optimal for Llama and Qwen models, and larger values (e.g., 0.6) are optimal for Gemma 3 models and Rnj-1.

Step 2-2: Latent Binary Factorization (LB-ADMM). We formulate initialization as finding factors $\mathbf{U}$ and $\mathbf{V}$ that approximate the preconditioned target $\widetilde{\mathbf{W}}_{\text{target}}$. To handle the non-convex landscape, we employ ADMM with ridge regularization λ , introducing auxiliary variables $\mathbf{Z}$ and scaled dual variables $\mathbf{\Lambda}$ to decouple constraints. The optimization problem is defined as:

$$\begin{aligned} \min_{\mathbf{U}, \mathbf{V}, \mathbf{Z}_{\mathbf{U}}, \mathbf{Z}_{\mathbf{V}}} \quad & \frac{1}{2} \|\widetilde{\mathbf{W}}_{\text{target}} - \mathbf{U}\mathbf{V}^\top\|_F^2 + \frac{\lambda}{2} (\|\mathbf{U}\|_F^2 + \|\mathbf{V}\|_F^2) \\ \text{s.t.} \quad & \mathbf{U} = \mathbf{Z}_{\mathbf{U}}, \mathbf{V} = \mathbf{Z}_{\mathbf{V}}. \end{aligned} \quad (4)$$

The solver alternates between updating continuous factors, auxiliary proxies, and dual variables. First, we update $\mathbf{U}$ (symmetrically $\mathbf{V}$) by solving a linear system regularized by penalty ρ and λ :

$$(\mathbf{V}^\top \mathbf{V} + (\rho + \lambda)\mathbf{I})\mathbf{U}^\top = \mathbf{V}^\top \widetilde{\mathbf{W}}_{\text{target}}^\top + \rho(\mathbf{Z}_{\mathbf{U}} - \mathbf{\Lambda}_{\mathbf{U}})^\top. \quad (5)$$

We employ stabilized Cholesky decomposition for this step, reducing the computational complexity to $\mathcal{O}(r^3/3)$ compared to general LU factorization, which scales as $\mathcal{O}(2r^3/3)$. This optimization is pivotal, as it enables NANOQUANT to scale efficiently to massive architectures (e.g., Llama-2-70B) within limited computational budgets.

Second, we update the auxiliary variable $\mathbf{Z}$ using the consensus variable $\mathbf{P}_{\mathbf{U}} \triangleq \mathbf{U} + \mathbf{\Lambda}$. We apply Sign-Value Independent Decomposition (SVID) (Pouransari et al., 2020; Xu

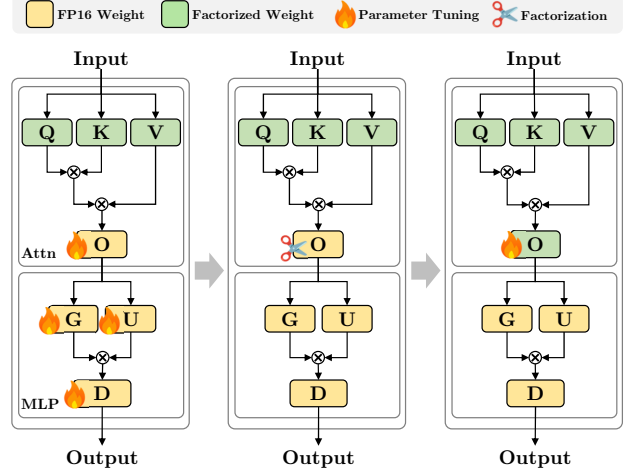


Figure 3. The NANOQUANT block reconstruction pipeline for compressing linear layers. The process sequentially optimizes each transformer block through three key phases: (1) **Error Propagation Mitigation** to adjust full-precision weights for accumulated errors; (2) **Low-Rank Binary Initialization**, which utilizes Latent Binary ADMM (LB-ADMM) to precisely generate latent binary factors and scales; and (3) **Factorized Component Refinement**, which fine-tunes the continuous latent matrices and scales using Straight-Through Estimators (STE) before final packing.

et al., 2024) to derive the optimal rank-1 approximation that preserves the sign structure:

$$\mathbf{Z}_{\mathbf{U}}^{(k+1)} = \text{SVID}(\mathbf{P}_{\mathbf{U}}^{(k+1)}). \quad (6)$$

Finally, we update the dual variables to enforce consensus, computed as $\mathbf{\Lambda}_{\mathbf{U}}^{(k+1)} = \mathbf{\Lambda}_{\mathbf{U}}^{(k)} + \mathbf{U}^{(k+1)} - \mathbf{Z}_{\mathbf{U}}^{(k+1)}$.

Step 2-3: Latent Magnitude Balancing. Upon convergence of ADMM, the pre-binary variables $\mathbf{P}_{\mathbf{U}}^{(K)}$ and $\mathbf{P}_{\mathbf{V}}^{(K)}$ possess inherent scale ambiguity, resulting in ill-conditioned proxies. To rectify this, we first recover the unscaled continuous proxies, defined as $\widehat{\mathbf{U}} = \widetilde{\mathbf{D}}_{\text{out}}^{-1} \mathbf{P}_{\mathbf{U}}^{(K)}$ and $\widehat{\mathbf{V}} = \widetilde{\mathbf{D}}_{\text{in}}^{-1} \mathbf{P}_{\mathbf{V}}^{(K)}$, and compute an equilibrium factor η to equalize their Frobenius norms:

$$\eta = \sqrt{\|\widehat{\mathbf{V}}\|_F / \|\widehat{\mathbf{U}}\|_F}. \quad (7)$$

The scaling vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ are computed directly from the balanced projections of these proxies to capture the magnitude information via the mean absolute value:

$$[\mathbf{s}_1]_i = \text{mean}(|\eta \widehat{\mathbf{u}}_i|), \quad [\mathbf{s}_2]_j = \text{mean}(|\eta^{-1} \widehat{\mathbf{v}}_j|), \quad (8)$$

where $\widehat{\mathbf{u}}_i$ and $\widehat{\mathbf{v}}_j$ denote the row vectors of $\widehat{\mathbf{U}}$ and $\widehat{\mathbf{V}}$, respectively. Following scale extraction, we define the final latent variables $\mathcal{U}$ and $\mathcal{V}$ to serve as well-conditioned initializers for the subsequent fine-tuning phase:

$$\begin{aligned} \mathcal{U} &:= \eta \widehat{\mathbf{U}} = \eta \widetilde{\mathbf{D}}_{\text{out}}^{-1} \mathbf{P}_{\mathbf{U}}^{(K)}, \\ \mathcal{V} &:= \eta^{-1} \widehat{\mathbf{V}} = \eta^{-1} \widetilde{\mathbf{D}}_{\text{in}}^{-1} \mathbf{P}_{\mathbf{V}}^{(K)}. \end{aligned} \quad (9)$$

This separation allows the explicit scales to handle magnitude at the input and output boundaries, ensuring that the core linear transformation proceeds sequentially without intervening scalar operations, thereby reducing computational overhead on hardware accelerators.

Step 3: Factorized Component Refinement. Following initialization, we refine the factorized components of the current target linear layer to align with the full-precision block outputs. Unlike approaches that defer binary optimization to a global stage (Boža & Macko, 2026) through PV-tuning (Malinovskii et al., 2024), we locally optimize these parameters during the block reconstruction phase. We jointly tune the continuous latent proxies $\mathcal{U}, \mathcal{V}$ and the scaling vectors $\mathbf{s}_1, \mathbf{s}_2$ using the Straight-Through Estimator (STE) (Bengio et al., 2013). Let $\mathcal{B}(\cdot)$ and $\widehat{\mathcal{B}}(\cdot)$ denote the full-precision and quantized mappings of the current transformer block (with all previously processed blocks fixed), respectively. The optimization objective is formulated as:

$$\min_{\mathcal{U}, \mathcal{V}, \mathbf{s}_1, \mathbf{s}_2} \|\mathcal{B}(\mathbf{X}_{\text{in}}) - \widehat{\mathcal{B}}(\mathbf{X}_{\text{in}}; \text{sign}(\mathcal{U}), \text{sign}(\mathcal{V}), \mathbf{s}_1, \mathbf{s}_2)\|_F^2. \quad (10)$$

This formulation allows gradients to propagate through the quantization function, enabling local identification of optimal sign structures while concurrently adjusting channel-wise magnitudes. Upon convergence, we fix $\mathbf{U}_{\pm 1} = \text{sign}(\mathcal{U})$ and $\mathbf{V}_{\pm 1} = \text{sign}(\mathcal{V})$ as the final binary values, and pack the binary weights into integer values.

3.3. Model Reconstruction

With the block-wise optimization concluded, the binary parameters are frozen and packed into efficient integer formats. Consequently, the final model reconstruction phase focuses exclusively on optimizing the floating-point scaling vectors $\mathbf{S}_{\text{global}} = \{\mathbf{s}_1, \mathbf{s}_2\}_{\forall l}$ to align the predictive distributions of the quantized model with those of the full-precision model (Kwon et al., 2022). The objective function minimizes the Kullback-Leibler (KL) divergence:

$$\min_{\mathbf{S}_{\text{global}}} D_{\text{KL}}(\text{softmax}(z_{\mathcal{M}}/T) \parallel \text{softmax}(z_{\widehat{\mathcal{M}}}/T)), \quad (11)$$

where $z_{\mathcal{M}} = \text{Logits}(M(X))$ and $z_{\widehat{\mathcal{M}}} = \text{Logits}(\widehat{M}(X; \mathbf{S}_{\text{global}}))$. Unlike prior methods that require extensive memory resources for global fine-tuning (Chen et al., 2025), this approach maintains fixed bit-packed binary weights throughout the process. This constraint substantially reduces the memory footprint, and it makes calibration of massive models, such as Llama-2-70B, feasible on a single GPU.

Algorithm 1 The NANOQUANT algorithm.

Input: FP teacher $\mathcal{M}$, calibration set $\mathcal{X}_{\text{cal}}$, rank r , robust-diagonal parameters (τ, γ) , ADMM parameters $(K, \rho, \lambda, \epsilon)$, optimization steps $(T_{\text{pre}}, T_{\text{post}}, T_{\text{glob}})$

Output: Quantized model $\widehat{\mathcal{M}}$ with packed binaries $\{\mathbf{U}_{\pm 1}^{(\ell)}, \mathbf{V}_{\pm 1}^{(\ell)}\}$ and float scales $\{\mathbf{s}_1^{(\ell)}, \mathbf{s}_2^{(\ell)}\}$

```

1: # Phase 1: Global Calibration
2: for each linear layer  $\ell$  do
3:   Run  $\mathcal{X}_{\text{cal}}$  through  $\mathcal{M}$  and collect statistics  $(\mathbf{z}_{\text{in}}, \mathbf{z}_{\text{out}})^{(\ell)}$ 
4:    $(\widehat{\mathbf{D}}_{\text{in}}, \widehat{\mathbf{D}}_{\text{out}})^{(\ell)} \leftarrow \text{ROBUSTDIAG}(\mathbf{z}_{\text{in}}^{(\ell)}, \mathbf{z}_{\text{out}}^{(\ell)}, \tau, \gamma)$ 
5: end for
6: # Phase 2: Block Reconstruction Pipeline
7:  $\widehat{\mathcal{M}} \leftarrow \mathcal{M}$  ▷ Init with teacher weights
8: for block  $b = 1, \dots, B$  do
9:    $\mathbf{X}_b \leftarrow \widehat{\mathcal{M}}_{<b}(\mathcal{X}_{\text{cal}})$  ▷ input activation after already-compressed prefix
10:   $\mathbf{Y}_b \leftarrow \mathcal{B}_b^{\text{FP}}(\mathbf{X}_b)$  ▷ teacher output for current block  $b$ 
11:  ▷ Step 1: Error Propagation Mitigation
12:   $\text{TUNEFP}(b, \mathbf{X}_b, \mathbf{Y}_b; T_{\text{pre}})$ 
13:  ▷ Step 2: Low-Rank Binary Initialization
14:  for linear layer  $\ell \in b$  with weight  $\mathbf{W}^{(\ell)}$  do
15:     $\widetilde{\mathbf{W}}^{(\ell)} \leftarrow \widehat{\mathbf{D}}_{\text{out}}^{(\ell)} \mathbf{W}^{(\ell)} \widehat{\mathbf{D}}_{\text{in}}^{(\ell)}$ 
16:     $(\mathbf{U}, \mathbf{V}, \mathbf{s}_1, \mathbf{s}_2)^{(\ell)} \leftarrow \text{LB-ADMM}(\widetilde{\mathbf{W}}^{(\ell)}; r, K, \rho, \lambda, \epsilon)$ 
17:  end for
18:  ▷ Step 3: Factorized Component Refinement
19:   $\text{TUNELATENTSTE}(b, \mathbf{X}_b, \mathbf{Y}_b; T_{\text{post}})$ 
20:  for linear layer  $\ell \in b$  do
21:     $\mathbf{U}_{\pm 1}^{(\ell)} \leftarrow \text{sign}(\mathbf{U}^{(\ell)}); \mathbf{V}_{\pm 1}^{(\ell)} \leftarrow \text{sign}(\mathbf{V}^{(\ell)})$ 
22:     $\text{PACKBINARY}(\mathbf{U}_{\pm 1}^{(\ell)}, \mathbf{V}_{\pm 1}^{(\ell)})$ 
23:  end for
24: end for
25: # Phase 3: Scale-Only Model Reconstruction
26: ▷ packed binaries are frozen during scale tuning
27:  $\text{TUNESCALESKD}(\widehat{\mathcal{M}}, \mathcal{M}, \mathcal{X}_{\text{cal}}; T_{\text{glob}})$ 
28: return  $\widehat{\mathcal{M}}$ 
    
```

4. Experiments

4.1. Experimental Setup

Implementation and Environment. The implementation of NANOQUANT relies on PyTorch (Paszke et al., 2019) and the Transformers library (Wolf et al., 2020). Primary quantization and evaluation experiments used a single NVIDIA H100 (80 GB) GPU to ensure consistency across model scales up to the 70B parameter regime. To assess deployment viability in resource-constrained environments, inference latency and memory footprints were analyzed on consumer-grade hardware, specifically an NVIDIA RTX 3050 (8 GB) GPU, and an edge device, an NVIDIA Jetson TX2.

Models and Datasets. Evaluations included a diverse set of LLM families, including Llama-2 (Touvron et al., 2023), Llama-3 (Grattafiori et al., 2024), Gemma 3 (Team et al., 2025), Qwen3 (Yang et al., 2025), and Rnj-1 (Essential AI, 2025), with sizes ranging from 0.6B to 70B parameters. This

Table 2. WikiText-2 perplexity ($\downarrow$) results of 1-bit and sub-1-bit post-training quantization methods. The evaluation encompasses pre-trained models from the Llama-2 (L2), Llama-3 (L3), Gemma 3 (G3), Qwen3 (Q3), and Rnj-1 (R1) families. In these abbreviations, the numerical suffix denotes the parameter count in billions (e.g., L3-8 represents Llama-3-8B). NANOQUANT demonstrates performance competitive with higher-bit baselines across these architectures.

W Bits	Total Bits	Method	L2-7	L2-13	L2-70	L3-1	L3-3	L3-8	L3-70	G3-1	G3-4	G3-12	G3-27	Q3-0.6	Q3-1.7	Q3-4	Q3-8	Q3-14	R1-8
16.00	-	-	5.47	4.88	3.32	9.74	7.81	6.24	2.86	10.60	7.39	5.86	4.88	12.66	9.39	7.89	7.00	6.37	8.19
1.00	1.00	RTN	1.63e5	4.82e4	1.57e5	5.39e8	1.82e13	4.41e5	3.98e5	3.64e22	2.96e17	1.90e24	6.29e21	2.58e7	5.14e8	1.45e6	5.12e6	7.05e9	7.26e5
	1.00	XNOR	6.59e4	9.80e3	1.37e4	1.15e5	1.78e6	8.50e5	8.61e5	1.91e8	4.50e6	5.00e6	3.32e6	3.27e7	1.60e6	1.56e10	1.37e8	1.63e8	6.25e4
	2.88	BiLLM	19.87	13.29	8.75	323.16	55.43	31.20	93.36	144.72	37.08	262.83	31.21	3.17e3	858.09	78.36	29.62	13.50	20.71
	4.13	STBLLM	10.12	8.08	5.26	187.40	25.46	16.68	155.43	80.54	21.97	63.45	16.46	329.76	1.32e3	35.03	20.04	10.72	14.78
	2.51	ARB-LLM _{RC}	11.80	8.43	5.20	66.36	23.87	19.06	7.89	67.43	23.37	32.47	16.80	129.52	49.51	18.04	12.74	10.25	15.13
	3.25	HBLLM _R	7.60	6.27	4.56	36.00	15.99	11.82	8.88	28.58	12.92	19.22	9.08	78.58	35.14	14.73	10.51	8.37	11.90
	1.00	NANOQUANT	10.34	8.71	6.52	25.59	17.90	14.97	11.32	35.30	19.64	24.70	27.21	27.56	19.21	14.29	12.47	10.92	15.45
0.80	4.00	STBLLM (6:8)	11.24	8.97	5.94	314.19	39.00	20.19	78.21	123.60	26.01	95.83	26.14	3.63e3	1.96e3	44.99	24.19	12.50	18.07
	0.80	NANOQUANT	12.20	10.14	7.61	33.08	22.09	18.16	13.75	50.15	25.38	32.84	28.83	33.79	25.31	19.33	14.83	12.88	20.06
0.55	3.50	STBLLM (4:8)	20.27	15.22	9.27	6.69e3	381.77	88.84	1.83e3	592.31	69.63	489.64	83.29	5.74e5	5.18e4	1.30e3	109.40	28.50	67.60
	0.55	NANOQUANT	16.66	13.46	9.82	49.01	32.33	25.69	19.69	78.22	40.69	45.29	32.98	52.94	33.74	32.86	20.04	17.06	32.62

Table 3. Zero-shot accuracy comparison on commonsense reasoning tasks using Llama-3 (L3) and Qwen3 (Q3) models. NANOQUANT maintains competitive accuracy against higher-bit binary PTQ baselines, despite utilizing a 1-bit representation.

Model	Bits	Method	ARC-e	ARC-c	BoolQ	Hella.	Wino.	PIQA	Avg.
L3-8	16.00	BF16	81.57	51.45	81.96	60.01	73.56	80.09	71.44
	4.13	STBLLM	36.87	19.97	48.01	36.47	57.62	61.48	39.83
	3.25	HBLLM _{col}	60.02	29.10	63.03	43.46	63.77	70.35	50.45
	2.88	BiLLM	36.32	18.34	56.36	30.16	51.93	57.56	38.16
	2.51	ARB-LLM _{RC}	49.71	22.95	64.28	34.73	56.04	63.28	44.23
	2.28	GPTQ(w2g64)	28.24	20.14	41.87	27.00	50.59	54.08	36.99
	1.00	NANOQUANT	43.69	20.31	61.47	33.81	55.96	60.45	45.95
Q3-8	16.00	BF16	81.52	52.47	83.03	58.81	72.38	79.11	71.22
	4.13	STBLLM	52.78	27.13	62.84	38.38	57.54	64.15	46.15
	3.25	HBLLM _{col}	68.43	36.77	68.53	45.68	63.85	71.65	54.30
	2.88	BiLLM	28.96	22.35	62.23	32.42	51.30	55.01	38.18
	2.51	ARB-LLM _{RC}	68.18	34.64	66.30	40.83	59.12	68.17	51.86
	2.28	GPTQ(w2g64)	28.62	20.99	43.64	29.52	50.04	54.68	37.92
	1.00	NANOQUANT	49.45	24.32	62.17	36.34	58.01	63.32	48.94

range addresses the sensitivity of smaller models to quantization noise (Li et al., 2020; Gong et al., 2024) and challenges the compression latency limits of larger architectures. Calibration used 128 samples from the WikiText-2 dataset (Merity et al., 2016) with a sequence length of 2048. Evaluation metrics included perplexity for next-token prediction and zero-shot accuracy across six commonsense reasoning tasks: WinoGrande (Sakaguchi et al., 2021), HellaSwag (Zellers et al., 2019), BoolQ (Clark et al., 2019), ARC-Easy, ARC-Challenge (Clark et al., 2018), and PIQA (Bisk et al., 2020). To evaluate the robustness of NANOQUANT across different data distributions, we also conduct experiments using the C4 dataset (Raffel et al., 2020), as detailed in Appendix D.

Baselines. We benchmark NANOQUANT against state-of-the-art binary post-training quantization (PTQ) methods, specifically BiLLM (Huang et al., 2024), ARB-LLM (Li et al., 2025), STBLLM (Dong et al., 2025), and HBLLM (Chen et al., 2026). Comparisons also include binary quantization-aware training (QAT) methods such as OneBit (Xu et al., 2024), BinaryMoS (Jo et al., 2024), LittleBit (Lee et al., 2025a), and DBF (Boža & Macko, 2026). We

Table 4. Comparing the compression and resource efficiency of various quantization methods, when compressing Llama-2-7B on NVIDIA H100 GPUs. NANOQUANT requires orders of magnitude less data and over an order of magnitude less GPU time to achieve binary quantization.

Method	PTQ/QAT	Bits	Model Size	Data	GPU Hours	PPL ($\downarrow$)
Full-Precision			13.48 GB			5.47
GPTQ (W2g64)	PTQ	2.28	2.37 GB	0.26M	0.1	21.00
STBLLM	PTQ	4.13	4.07 GB	0.26M	0.9	10.12
HBLLM _R	PTQ	3.25	3.16 GB	0.26M	2.2	7.60
BiLLM	PTQ	2.88	2.85 GB	0.26M	1.1	19.87
ARB-LLM _{RC}	PTQ	2.51	2.55 GB	0.26M	1.3	11.80
OneBit	QAT	1.04	1.37 GB	155.46M	700.7	9.73
BinaryMoS	QAT	1.08	1.40 GB	196.00M	92.8	7.88
DBF	QAT	1.00	1.33 GB	1.38B	37.6	9.25
LittleBit	QAT	1.04	1.33 GB	196.00M	123.6	9.08
NANOQUANT	PTQ	1.00	1.33 GB	0.26M	1.7	10.34
NANOQUANT	PTQ	1.00	1.33 GB	2.10M	2.5	8.85

utilize official open-source implementations for PTQ baselines and select the highest-performing variants, such as ARB-LLM_{RC} and HBLLM_{col}. Regarding QAT methods, we report the performance metrics for OneBit and BinaryMoS directly from their original literature. Conversely, we reproduce specific components of DBF and LittleBit to validate initialization strategies.

4.2. Accuracy Analysis

Next Token Prediction. Table 2 presents the perplexity comparison between NANOQUANT and existing binary PTQ baselines. The results indicate that NANOQUANT maintains functional perplexity across diverse model families while using fewer bits than competing methods. Prior binary PTQ approaches often struggle to break the 1-bit barrier due to structural overhead, but NANOQUANT achieves sub-1-bit compression without a catastrophic degradation in the predictive distribution. This finding suggests that the proposed low-rank factorization effectively captures the salient weight information required for language modeling, even at extreme compression rates.

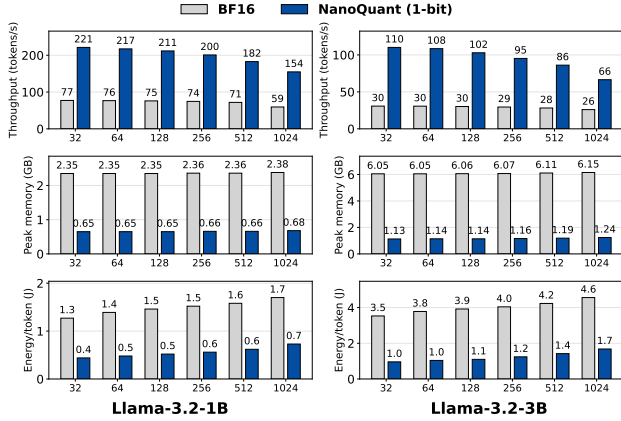


Figure 4. On an NVIDIA RTX 3050 (8 GB) GPU, NANOQUANT delivers up to $3.6\times$ higher decoding throughput, $5.4\times$ lower peak memory usage, and $3.9\times$ greater energy efficiency compared to BF16 baselines for Llama-3.2-1B and 3B models.

Zero-Shot Reasoning. The evaluation of commonsense reasoning tasks in Table 3 reveals that NANOQUANT yields performance competitive with higher-bit binary PTQ baselines. Furthermore, the method approaches the zero-shot accuracy of binary QAT methods. This result is notable because QAT approaches typically require extensive training on billions of tokens. In contrast, NANOQUANT achieves comparable fidelity using orders of magnitude less data and compute. This efficiency suggests that precise initialization and block-wise reconstruction may substitute for the expensive end-to-end retraining traditionally required for binary quantization.

4.3. Compression vs. Model Size

We analyze the storage efficiency of various quantization paradigms using Llama-2-7B as a case study, as shown in Table 4. The analysis reveals that standard binary PTQ methods often incur substantial overhead due to auxiliary parameters. Consequently, their effective storage requirements exceed those of 2-bit quantization methods such as GPTQ (Frantar et al., 2022). For instance, BiLLM and STBLLM require 2.88 and 4.13 bits per weight, respectively, whereas GPTQ (W2g64) uses only 2.28 bits. NANOQUANT overcomes this limitation and achieves genuine 1-bit and sub-1-bit post-training quantization. By minimizing meta-data overhead, it offers a storage solution that is strictly more efficient than existing PTQ baselines while maintaining perplexity levels competitive with resource-intensive QAT methods. A comprehensive breakdown of the storage requirements and the mathematical derivation of effective bits per weight (BPW) for both NANOQUANT and existing baselines can be found in Appendix F.

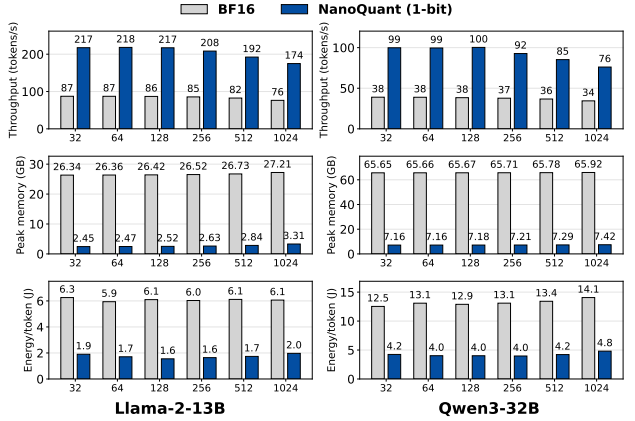


Figure 5. Datacenter inference efficiency on a single NVIDIA H100 (80 GB) GPU. NANOQUANT enables faster decoding throughput while maintaining superior memory and energy efficiency for Llama-2-13B and Qwen3-32B, compared to the PyTorch BF16 baseline.

4.4. Inference Efficiency

The extreme compression ratio achieved by NANOQUANT translates directly into reduced memory footprints and enhanced throughput, particularly in memory-bound regimes. To quantify this, we compared the decoding performance of NANOQUANT against a PyTorch BF16 baseline. We focused on this comparison because optimized inference kernels for the binary PTQ baselines in Table 2 are currently unavailable.

Consumer Hardware. On an NVIDIA RTX 3050 (8 GB) GPU, NANOQUANT enables up to a $3.6\times$ speedup in inference throughput for Llama-3.2-3B. Additionally, the method achieves a $5.4\times$ reduction in peak memory usage and a $3.9\times$ improvement in energy efficiency per token, as shown in Figure 4. Beyond speed, the method fundamentally expands accessibility. NANOQUANT compresses the Llama-2-70B model from 137.95 GB to 5.75 GB, representing a $24\times$ compression factor. This reduction allows a 70B parameter model to fit entirely within the VRAM of a consumer-grade 8 GB GPU and effectively lowers the barrier to entry for large-scale model deployment. To evaluate deployment viability in even more constrained environments, we extended our analysis to embedded systems. Detailed performance metrics on the NVIDIA Jetson TX2 are provided in Appendix E.

Datacenter Hardware. On high-end hardware (NVIDIA H100 (80 GB) GPU), NANOQUANT alleviates memory bandwidth bottlenecks and demonstrates up to $10\times$ lower memory usage during inference. As illustrated in Figure 5, this results in faster single-batch inference and superior energy efficiency compared to the BF16 baseline. Additional

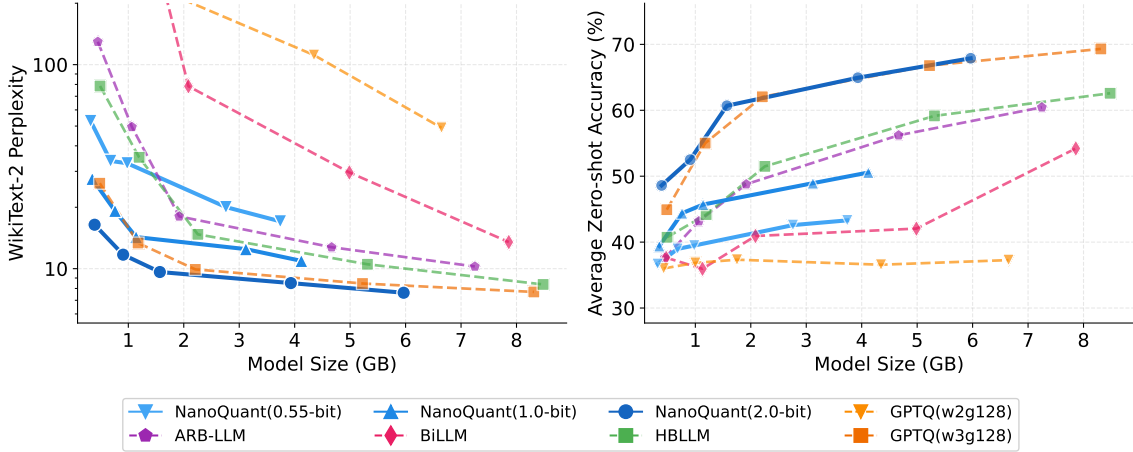


Figure 6. Pareto optimality analysis for models in the Qwen3 family (0.6B, 1.7B, 4B, 8B, 14B). NANOQUANT establishes a new efficiency frontier in the low-bit regime, offering superior accuracy-per-bit trade-offs compared to existing state-of-the-art binary PTQ methods.

Table 5. Initialization via latent-binary ADMM (LB-ADMM) from NANOQUANT outperforms other low-rank binary initialization strategies, when compressing Rnj-1 (Essential AI, 2025) to 0.8 bits.

Initialization Method	PPL ($\downarrow$)	Zero-shot ($\uparrow$)
Dual-SVID (Lee et al., 2025a)	167.73	35.11
DBF ADMM (Boža & Macko, 2026)	30.27	37.20
LB-ADMM (Ours)	20.06	39.29

Table 6. Component-wise efficacy analysis of the NANOQUANT pipeline on Qwen3-8B-Base. The table demonstrates the contribution of each module—Initialization, Error Mitigation, Factorized Component Refinement, and Model Reconstruction—towards enhancing performance when combined.

Block Reconstruction			Model Recon.	PPL ($\downarrow$)	Zero-Shot ($\uparrow$)
Initialization	Error Mitigation	Fact. Refinement			
✓	✗	✗	✗	206.03	36.89
✓	✓	✗	✗	15.07	46.40
✓	✗	✓	✗	15.00	47.88
✓	✓	✓	✗	13.58	46.75
✓	✓	✓	✓	12.47	48.94

kernel implementation details are provided in Appendix E.

4.5. Ablation Studies

Initialization Strategy. We investigate the hypothesis that precise initialization of low-rank binary matrices is critical for convergence in PTQ. We integrated initialization strategies from prominent QAT methods, specifically LittleBit (Lee et al., 2025a) and DBF (Boža & Macko, 2026), into our reconstruction pipeline. Table 5 demonstrates that Latent-Binary ADMM (LB-ADMM) outperforms these alternatives in both perplexity and zero-shot tasks. This result indicates that solving the combinatorial problem of binary factorization prior to fine-tuning provides a more stable optimization

Table 7. NANOQUANT achieves comparable performance with QAT methods DBF and LittleBit, while using orders of magnitude less data and compute time, when compressing Qwen3-4B-Base and Llama-2-7B to 1 bit.

Model	Method	Data	GPU Hours	PPL ($\downarrow$)	Zero-shot ($\uparrow$)
Q3-4B	LittleBit	169.50M	92.5	14.79	47.32
	DBF	1.19B	25.3	14.62	52.30
	NANOQUANT	1.05M	2.3	12.62	50.63
L2-7B	LittleBit	196.00M	123.6	9.08	54.92
	DBF	1.38B	37.6	9.25	54.24
	NANOQUANT	1.05M	2.1	9.01	51.01

landscape than the initialization schemes used in existing QAT frameworks.

Component Efficacy. Table 6 dissects the contribution of each algorithmic component. This improvement stems from the combination of robust initialization and block-wise reconstruction. Each module contributes distinctly to preserving the model’s representational capacity under extreme compression.

Pareto Optimality. Finally, we analyze the trade-off between model size and performance across the Qwen3 family, as shown in Figure 6. NANOQUANT establishes a new Pareto frontier in the low-memory regime and consistently outperforms previous binary PTQ baselines. These results suggest that low-rank binary representations are a viable alternative to low-bit integer quantization for memory-critical applications.

Comparison with Low-Rank Binary QAT. Table 7 contrasts NANOQUANT with state-of-the-art QAT methods. DBF (Boža & Macko, 2026) and LittleBit (Lee et al., 2025a) rely on training over 1 billion and 100 million tokens respectively. We find that with 512 calibration samples,

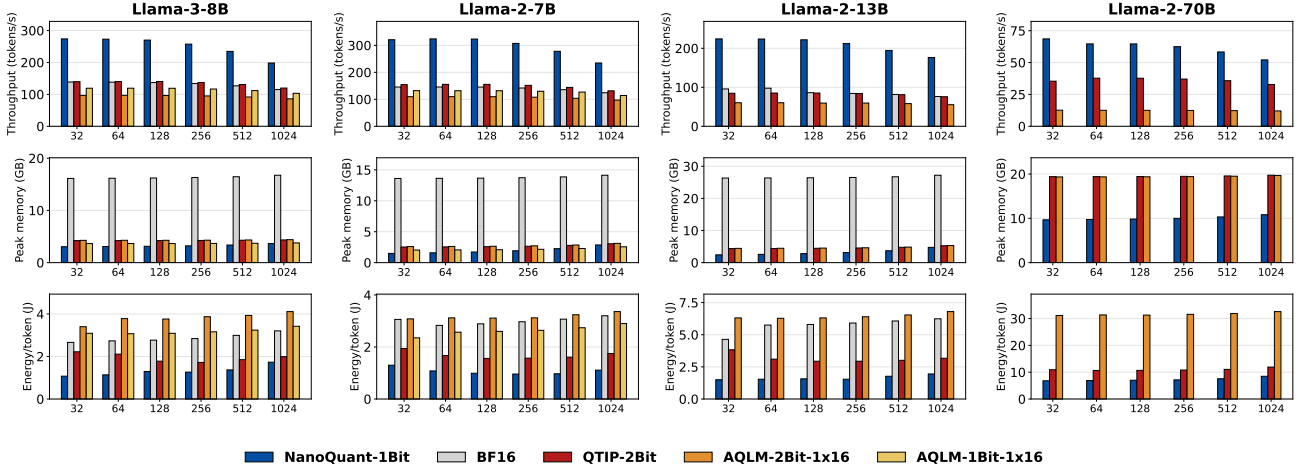


Figure 7. LLM decoding performance of NANOQUANT, compared with PyTorch BF16 and vector quantization methods (AQLM, QTIP) for 128 input tokens and various output sequence lengths, on an NVIDIA H100 (80 GB) GPU. NANOQUANT shows superior inference speed, memory efficiency, and energy efficiency, compared to vector quantization methods and BF16.

Table 8. Comparison of NANOQUANT with state-of-the-art vector quantization methods AQLM (Egiazarian et al., 2024), PV-Tuning (Malinovskii et al., 2024), and QTIP (Tseng et al., 2024b) for compressing Llama-2-7B. Zero-shot denotes the average over ARC-C, ARC-E, HellaSwag, PIQA, and WinoGrande. NANOQUANT achieves competitive performance under extreme compression while using substantially less calibration data.

Target	Method	Bits	Model Size	Data	PPL ($\downarrow$)	Zero-shot ($\uparrow$)
2-bit	QTIP	2.02	2.15 GB	12.58M	6.29	62.05
	AQLM	2.02	2.15 GB	8.00M	6.64	56.47
	AQLM + PV	2.02	2.15 GB	8.00M	5.84	61.35
	NANOQUANT	2.00	2.14 GB	1.05M	6.52	59.09
1.5-bit	AQLM + PV	1.58	1.81 GB	8.00M	7.32	55.22
	NANOQUANT	1.50	1.81 GB	1.05M	7.08	55.81
1-bit	AQLM + PV	1.02	1.34 GB	8.00M	8.28	50.21
	NANOQUANT	1.00	1.33 GB	1.05M	8.99	47.81

NANOQUANT achieves comparable predictive performance with binary QAT methods. This data efficiency validates the effectiveness of the proposed PTQ formulation for scenarios where full-scale retraining is impractical.

NANOQUANT vs Vector Quantization. We compare the performance of NANOQUANT with state-of-the-art 2-bit vector quantization methods (Egiazarian et al., 2024), PV-tuning (Malinovskii et al., 2024), and QTIP (Tseng et al., 2024b). These methods utilize significantly more data and compute than other low-bit PTQ methods, as in Table 8. Notably, compressing Llama-2-7B with NANOQUANT takes less than 3 hours on an NVIDIA H100 (80 GB) GPU, while AQLM is reported to take at least 1 day on multiple A100 GPUs, and PV-Tuning even longer. Nevertheless, NANOQUANT shows competitive performance with data- and compute-intensive vector quantization baselines, making NANOQUANT a practical compression method for resource-constrained environments.

4.6. Limitations and Future Work

Although our experiments demonstrate data efficiency using a small calibration set, scaling the data and compute budget could enhance performance on more complex reasoning tasks. Regarding inference, the custom CUDA kernels demonstrate promising results, as detailed in subsection 4.4. However, further optimization for next-generation architectures such as NVIDIA Blackwell GPUs or edge-specific hardware could yield additional speedups. Additionally, while NANOQUANT outperforms some 2-bit baselines, further enhancing capabilities to outperform higher-bit 2 or 3-bit PTQ performance remains an open challenge for the sub-binary regime. Future work will focus on optimizing the compression runtime, exploring the scalability of the method to larger calibration datasets, and investigating adaptive rank allocation across layers to further optimize the accuracy-per-bit Pareto frontier.

5. Conclusion

We propose NANOQUANT, an efficient and accurate post-training quantization method to enable 1-bit and sub-1-bit weight quantization of LLMs of up to 70B parameters, with only a single GPU. NANOQUANT enables rapid binarization and extremely compact weight storage, significantly reducing the memory footprint of LLM inference. Custom binary CUDA kernels further improve energy efficiency and decoding speed. Our approach achieves up to $24\times$ model compression, making it feasible to run a 70B-parameter LLM on an 8 GB GPU. NANOQUANT lowers the hardware barrier for some large-model inference by enabling fast, efficient compression and inference for researchers and developers in resource-constrained settings, and advances the frontier of extreme LLM quantization.

Acknowledgments

We thank Drs. Heonjae Ha and Sangjeong Lee for their guidance, support and insightful feedback throughout this project.

Impact Statement

This work presents NANOQUANT, a sub-1-bit post-training quantization algorithm for large language models (LLMs). By enabling the deployment of massive models (e.g., Llama-2-70B) on consumer hardware (e.g., a single 8 GB GPU) and edge devices, our method significantly lowers the barrier to entry for advanced AI research and application. This contributes to the democratization of AI, allowing individuals and institutions with limited computational resources to utilize state-of-the-art LLMs. Furthermore, NANOQUANT is resource- and energy-efficient, as it drastically reduces the memory bandwidth and energy consumption required for inference, as demonstrated by our energy-efficiency experiments.

References

- Arai, Y. and Ichikawa, Y. Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization. *Advances in Neural Information Processing Systems*, 38: 151916–151951, 2026.
- Barber, R. F. and Sidky, E. Y. Convergence for Non-convex ADMM, with Applications to CT Imaging. *Journal of Machine Learning Research*, 25(38):1–46, 2024. URL <http://jmlr.org/papers/v25/21-0831.html>.
- Bengio, Y., Léonard, N., and Courville, A. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *arXiv:1308.3432*, 2013.
- Bertsekas, D. P. *Constrained Optimization and Lagrange Multiplier Methods*. Academic Press, 2014.
- Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. PIQA: Reasoning about Physical Commonsense in Natural Language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- Boyd, S. and Vandenberghe, L. *Convex Optimization*. Cambridge University Press, 2004.
- Boyd, S., Parikh, N., Chu, E., Peleato, B., Eckstein, J., et al. Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers. *Foundations and Trends® in Machine Learning*, 3(1):1–122, 2011.
- Boža, V. and Macko, V. Addition Is Almost All You Need: Compressing Neural Networks with Double Binary Factorization. *Transactions on Machine Learning Research*, 2026.
- Chee, J., Cai, Y., Kuleshov, V., and De Sa, C. M. QuIP: 2-Bit Quantization of Large Language Models with Guarantees. *Advances in Neural Information Processing Systems*, 36: 4396–4429, 2023.
- Chen, M., Shao, W., Xu, P., Wang, J., Gao, P., Zhang, K., and Luo, P. EfficientQAT: Efficient Quantization-Aware Training for Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10081–10100, 2025.
- Chen, N., Ye, W., and Jiang, Y. HBLLM: Wavelet-Enhanced High-Fidelity 1-Bit Quantization for LLMs. *Advances in Neural Information Processing Systems*, 38:153595–153631, 2026.
- Cherniuk, D., Abukhovich, S., Phan, A.-H., Oseledets, I., Cichocki, A., and Gusk, J. Quantization Aware Factorization for Deep Neural Network Compression. *Journal of Artificial Intelligence Research*, 81:973–988, 2024.
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, 2019.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv Preprint arXiv:1803.05457*, 2018.
- Dong, P., Li, L., Zhong, Y., Du, D., Fan, R., Chen, Y., Tang, Z., Wang, Q., Xue, W., Guo, Y., et al. STBLLM: Breaking the 1-Bit Barrier with Structured Binary LLMs. In *International Conference on Learning Representations*, volume 2025, pp. 102882–102907, 2025.
- Egiazarian, V., Panferov, A., Kuznedelev, D., Frantar, E., Babenko, A., and Alistarh, D. Extreme Compression of Large Language Models via Additive Quantization. In *International Conference on Machine Learning*, pp. 12284–12303. PMLR, 2024.
- Essential AI. Rnj-1: Building Instruments of Intelligence, 2025. URL <https://essential.ai/research/rnj-1>. Research blog post, Essential AI.
- Frantar, E. and Alistarh, D. QMoE: Sub-1-Bit Compression of Trillion Parameter Models. *Proceedings of Machine Learning and Systems*, 6:439–451, 2024.

- Frantar, E., Ashkboos, S., Hoeffler, T., and Alistarh, D. GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers. *arXiv Preprint arXiv:2210.17323*, 2022.
- Frantar, E., Castro, R. L., Chen, J., Hoeffler, T., and Alistarh, D. MARLIN: Mixed-Precision Auto-Regressive Parallel Inference on Large Language Models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pp. 239–251, 2025.
- Froese, V. and Hertrich, C. Training Neural Networks is NP-Hard in Fixed Dimension. *Advances in Neural Information Processing Systems*, 36:44039–44049, 2023.
- Gabay, D. and Mercier, B. A Dual Algorithm for the Solution of Nonlinear Variational Problems via Finite Element Approximation. *Computers & Mathematics with Applications*, 2(1):17–40, 1976.
- Glowinski, R. and Marroco, A. Sur l’Approximation, par Éléments Finis d’Ordre Un, et la Résolution, par Pénalisation-Dualité d’une Classe de Problèmes de Dirichlet Non Linéaires. *Revue Française d’Automatique, Informatique, Recherche Opérationnelle. Analyse Numérique*, 9(R2):41–76, 1975.
- Gong, Z., Liu, J., Wang, J., Cai, X., Zhao, D., and Yan, R. What Makes Quantization for Large Language Models Hard? An Empirical Study from the Lens of Perturbation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 18082–18089, 2024.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The Llama 3 Herd of Models. *arXiv Preprint arXiv:2407.21783*, 2024.
- Gu, H., Li, L., Wang, Z., Liu, B., Zhu, Q., Han, S., and Guo, Y. BTC-LLM: Efficient Sub-1-Bit LLM Quantization via Learnable Transformation and Binary Codebook. *arXiv Preprint arXiv:2506.12040*, 2025.
- Hicham Badri, A. S. Gemlite: Towards Building Custom Low-Bit Fused CUDA Kernels, August 2024. URL https://mobiusml.github.io/gemlite_blog/.
- Holmes, C., Zhang, M., He, Y., and Wu, B. NxMTransformer: Semi-Structured Sparsification for Natural Language Understanding via ADMM. *Advances in Neural Information Processing Systems*, 34:1818–1830, 2021.
- Hong, M., Luo, Z.-Q., and Razaviyayn, M. Convergence Analysis of Alternating Direction Method of Multipliers for a Family of Nonconvex Problems. *SIAM Journal on Optimization*, 26(1):337–364, 2016.
- Huang, T., Singhanian, P., Sanjabi, M., Mitra, P., and Razaviyayn, M. Alternating Direction Method of Multipliers for Quantization. In *International Conference on Artificial Intelligence and Statistics*, pp. 208–216. PMLR, 2021.
- Huang, W., Liu, Y., Qin, H., Li, Y., Zhang, S., Liu, X., Magno, M., and Qi, X. BiLLM: Pushing the Limit of Post-Training Quantization for LLMs. In *International Conference on Machine Learning*, pp. 20023–20042. PMLR, 2024.
- Hubara, I., Nahshan, Y., Hanani, Y., Banner, R., and Soudry, D. Accurate Post Training Quantization with Small Calibration Sets. In *International Conference on Machine Learning*, pp. 4466–4475. PMLR, 2021.
- Jo, D., Kim, T., Kim, Y., et al. Mixture of Scales: Memory-Efficient Token-Adaptive Binarization for Large Language Models. *Advances in Neural Information Processing Systems*, 37:137474–137494, 2024.
- Kim, J., El Halabi, M., Park, W., Schaefer, C. J., Lee, D., Park, Y., Lee, J. W., and Song, H. O. GuidedQuant: Large Language Model Quantization via Exploiting End Loss Guidance. In *International Conference on Machine Learning*, pp. 30011–30037. PMLR, 2025.
- Kwon, S. J., Kim, J., Bae, J., Yoo, K. M., Kim, J.-H., Park, B., Kim, B., Ha, J.-W., Sung, N., and Lee, D. AlphaTuning: Quantization-Aware Parameter-Efficient Adaptation of Large-Scale Pre-Trained Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 3288–3305, 2022.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Ledoit, O. and Wolf, M. A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.
- Lee, B., Kim, D., You, Y., and Kim, Y. LittleBit: Ultra Low-Bit Quantization via Latent Factorization. *Advances in Neural Information Processing Systems*, 2025a.
- Lee, K., Jang, H., Lee, D., Alistarh, D., and Lee, N. The Unseen Frontier: Pushing the Limits of LLM Sparsity with Surrogate-Free ADMM. *arXiv Preprint arXiv:2510.01650*, 2025b.
- Leng, C., Dou, Z., Li, H., Zhu, S., and Jin, R. Extremely Low Bit Neural Network: Squeeze the Last Bit Out with ADMM. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

- Li, Z., Wallace, E., Shen, S., Lin, K., Keutzer, K., Klein, D., and Gonzalez, J. Train Large, Then Compress: Rethinking Model Size for Efficient Training and Inference of Transformers. In *International Conference on Machine Learning*, pp. 5958–5968. PMLR, 2020.
- Li, Z., Yan, X., Zhang, T., Qin, H., Xie, D., Tian, J., Kong, L., Zhang, Y., Yang, X., et al. ARB-LLM: Alternating Refined Binarizations for Large Language Models. In *International Conference on Learning Representations*, volume 2025, pp. 93900–93912, 2025.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.
- Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., and Blankevoort, T. SpinQuant: LLM Quantization with Learned Rotations. In *International Conference on Learning Representations*, volume 2025, pp. 92009–92032, 2025.
- Liu, Z., Zhao, C., Huang, H., Chen, S., Zhang, J., Zhao, J., Roy, S., Jin, L., Xiong, Y., Shi, Y., et al. ParetoQ: Scaling Laws in Extremely Low-Bit LLM Quantization. *Advances in Neural Information Processing Systems*, 38: 91311–91336, 2026.
- Makni, M., Meng, X., and Mazumder, R. 3BASiL: An Algorithmic Framework for Sparse Plus Low-Rank Compression of LLMs. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=byNNv5Et10>.
- Malinovskii, V., Mazur, D., Ilin, I., Kuznedelev, D., Burlachenko, K., Yi, K., Alistarh, D., and Richtarik, P. PV-Tuning: Beyond Straight-Through Estimation for Extreme LLM Compression. *Advances in Neural Information Processing Systems*, 37:5074–5121, 2024.
- Martens, J. and Grosse, R. Optimizing Neural Networks with Kronecker-Factored Approximate Curvature. In *International Conference on Machine Learning*, pp. 2408–2417. PMLR, 2015.
- Meng, X., Behdin, K., Wang, H., and Mazumder, R. ALPS: Improved Optimization for Highly Sparse One-Shot Pruning for Large Language Models. *Advances in Neural Information Processing Systems*, 2024.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer Sentinel Mixture Models, 2016.
- Nagel, M., Amjad, R. A., Van Baalen, M., Louizos, C., and Blankevoort, T. Up or Down? Adaptive Rounding for Post-Training Quantization. In *International Conference on Machine Learning*, pp. 7197–7206. PMLR, 2020.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32, 2019.
- Pouransari, H., Tu, Z., and Tuzel, O. Least Squares Binary Quantization of Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 698–699, 2020.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21 (140):1–67, 2020.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Shao, W., Chen, M., Zhang, Z., Xu, P., Zhao, L., Li, Z., Zhang, K., Peng, G., Qiao, Y., and Luo, P. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models. In *International Conference on Learning Representations*, volume 2024, pp. 45472–45496, 2024.
- Team, G., Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., et al. Gemma 3 Technical Report. *arXiv Preprint arXiv:2503.19786*, 2025.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv Preprint arXiv:2307.09288*, 2023.
- Tseng, A., Chee, J., Sun, Q., Kuleshov, V., and De Sa, C. QuIP #: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. In *International Conference on Machine Learning*, pp. 48630–48656. PMLR, 2024a.
- Tseng, A., Sun, Q., Hou, D., and De Sa, C. M. QTIP: Quantization with Trellises and Incoherence Processing. *Advances in Neural Information Processing Systems*, 37: 59597–59620, 2024b.
- Wang, H., Ma, S., Dong, L., Huang, S., Wang, H., Ma, L., Yang, F., Wang, R., Wu, Y., and Wei, F. BitNet: Scaling 1-Bit Transformers for Large Language Models. *arXiv Preprint arXiv:2310.11453*, 2023.
- Wang, Y., Yin, W., and Zeng, J. Global Convergence of ADMM in Nonconvex Nonsmooth Optimization. *Journal of Scientific Computing*, 78(1):29–63, 2019.

- Wijmans, E., Huval, B., Hertzberg, A., Koltun, V., and Krähenbühl, P. Cut Your Losses in Large-Vocabulary Language Models. In *International Conference on Learning Representations*, volume 2025, pp. 68174–68193, 2025.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, 2020.
- Xu, J., Chen, X., Hu, S., Yu, J., Liu, X., and Meng, H. Low-Bit Quantization of Recurrent Neural Network Language Models Using Alternating Direction Methods of Multipliers. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 7939–7943. IEEE, 2020.
- Xu, J., Yu, J., Hu, S., Liu, X., and Meng, H. Mixed Precision Low-Bit Quantization of Neural Network Language Models for Speech Recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3679–3693, 2021.
- Xu, Y., Han, X., Yang, Z., Wang, S., Zhu, Q., Liu, Z., Liu, W., and Che, W. OneBit: Towards Extremely Low-Bit Large Language Models. *Advances in Neural Information Processing Systems*, 37:66357–66382, 2024.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 Technical Report. *arXiv Preprint arXiv:2505.09388*, 2025.
- Yang, Y., Jia, Q.-S., Xu, Z., Guan, X., and Spanos, C. J. Proximal ADMM for Nonconvex and Nonsmooth Optimization. *Automatica*, 146:110551, 2022.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. HellaSwag: Can a Machine Really Finish Your Sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, 2019.
- Zhao, J., Zhang, M., Wang, M., Shang, Y., Zhang, K., Guan, W., Wang, Y., and Zhang, M. PTQ1.61: Push the Real Limit of Extremely Low-Bit Post-Training Quantization Methods for Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4483–4502, 2025.
- Zheng, L., Yin, L., Xie, Z., Sun, C. L., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., et al. SGLang: Efficient Execution of Structured Language Model Programs. *Advances in Neural Information Processing Systems*, 37:62557–62583, 2024.

A. Theoretical Analysis: Magnitude Balancing

In this section, we justify the magnitude balancing step used in NANOQUANT. The low-rank factorization $\mathbf{W} \approx \mathcal{U}\mathcal{V}^\top$ is scale-invariant: for any scalar $\eta > 0$,

$$\mathcal{U}\mathcal{V}^\top = (\eta\mathcal{U})(\eta^{-1}\mathcal{V})^\top. \quad (12)$$

Thus, the same reconstructed weight matrix can be represented by infinitely many pairs of latent factors with different relative magnitudes. Without controlling this ambiguity, one factor can become excessively large while the other becomes excessively small, which can lead to poorly conditioned updates and unstable scale extraction. Magnitude balancing selects a representative from this equivalent family whose two factors have comparable Frobenius norms before extracting the channel-wise scales.

In our formulation, the continuous latent variables are decomposed into channel-wise scales and binary directions:

$$\mathcal{U} \approx \text{diag}(\mathbf{s}_1)\mathbf{U}_{\pm 1}, \quad \mathcal{V} \approx \text{diag}(\mathbf{s}_2)\mathbf{V}_{\pm 1}, \quad (13)$$

where $\mathbf{s}_1 \in \mathbb{R}^m$ and $\mathbf{s}_2 \in \mathbb{R}^n$ capture output-channel and input-channel magnitudes, respectively.

A.1. Magnitude Balancing under Scale Ambiguity

Proposition 1 (Balanced representative under scale ambiguity). *Let $\hat{\mathcal{U}}$ and $\hat{\mathcal{V}}$ be nonzero latent factors obtained after LB-ADMM. Among all equivalent rescalings*

$$\mathcal{U}_\eta = \eta\hat{\mathcal{U}}, \quad \mathcal{V}_\eta = \eta^{-1}\hat{\mathcal{V}}, \quad \eta > 0, \quad (14)$$

which preserve $\mathcal{U}_\eta\mathcal{V}_\eta^\top = \hat{\mathcal{U}}\hat{\mathcal{V}}^\top$, the value

$$\eta^* = \sqrt{\frac{\|\hat{\mathcal{V}}\|_F}{\|\hat{\mathcal{U}}\|_F}} \quad (15)$$

minimizes the total factor magnitude

$$\mathcal{J}(\eta) = \frac{1}{2} \left(\|\eta\hat{\mathcal{U}}\|_F^2 + \|\eta^{-1}\hat{\mathcal{V}}\|_F^2 \right). \quad (16)$$

For this minimum-energy representative, the balanced factors satisfy

$$\|\mathcal{U}_{\eta^*}\|_F = \|\mathcal{V}_{\eta^*}\|_F. \quad (17)$$

Proof. Expanding the objective gives

$$\mathcal{J}(\eta) = \frac{1}{2} \left(\eta^2 \|\hat{\mathcal{U}}\|_F^2 + \eta^{-2} \|\hat{\mathcal{V}}\|_F^2 \right). \quad (18)$$

Differentiating with respect to η yields

$$\frac{d\mathcal{J}}{d\eta} = \eta \|\hat{\mathcal{U}}\|_F^2 - \eta^{-3} \|\hat{\mathcal{V}}\|_F^2. \quad (19)$$

Setting the derivative to zero gives

$$\eta^4 = \frac{\|\hat{\mathcal{V}}\|_F^2}{\|\hat{\mathcal{U}}\|_F^2}, \quad (20)$$

and therefore

$$\eta^* = \sqrt{\frac{\|\hat{\mathcal{V}}\|_F}{\|\hat{\mathcal{U}}\|_F}}. \quad (21)$$

Substituting η^* into the two factor norms gives

$$\|\eta^*\hat{\mathcal{U}}\|_F^2 = \|\hat{\mathcal{U}}\|_F \|\hat{\mathcal{V}}\|_F = \|(\eta^*)^{-1}\hat{\mathcal{V}}\|_F^2, \quad (22)$$

which proves that the selected rescaling equalizes the Frobenius norms of the two factors. $\square$

This result directly motivates the magnitude balancing step in NANOQUANT. After LB-ADMM, we recover the unscaled latent factors and apply

$$\mathcal{U} \leftarrow \eta^* \hat{\mathcal{U}}, \quad \mathcal{V} \leftarrow (\eta^*)^{-1} \hat{\mathcal{V}}. \quad (23)$$

The channel-wise scales s_1 and s_2 are then extracted from these balanced factors. This prevents one side of the factorization from absorbing most of the magnitude, which would otherwise make the subsequent scale extraction sensitive to numerical range and rounding effects.

A.2. Conditioning Intuition

Magnitude balancing also improves numerical stability. The ADMM update for one factor involves solving a linear system with a matrix of the form

$$\mathbf{H} = \mathbf{V}^\top \mathbf{V} + (\rho + \lambda) \mathbf{I}, \quad (24)$$

where ρ is the ADMM penalty and λ is the ridge regularization coefficient. The condition number is

$$\kappa(\mathbf{H}) = \frac{\sigma_{\max}(\mathbf{V})^2 + \rho + \lambda}{\sigma_{\min}(\mathbf{V})^2 + \rho + \lambda}. \quad (25)$$

The positive diagonal shift $(\rho + \lambda) \mathbf{I}$ ensures that the system is positive definite, but the scale ambiguity of the factorization can still create undesirable numerical regimes. If $\|\mathbf{V}\|_F$ becomes extremely large, the conditioning of $\mathbf{H}$ increasingly reflects the conditioning of the Gram matrix $\mathbf{V}^\top \mathbf{V}$. If $\|\mathbf{V}\|_F$ becomes extremely small, the data-dependent terms in the update become weak relative to the penalty and regularization terms.

By selecting the balanced representative of the equivalent factorization, NANOQUANT avoids these extreme regimes before the subsequent refinement stage. Thus, magnitude balancing does not change the reconstructed matrix $\mathcal{U}\mathcal{V}^\top$, but it provides a better-conditioned parameterization for scale extraction and straight-through estimator refinement.

B. Stability Analysis of Low-Rank Binary Initialization

This section analyzes the numerical stability of the robust Hessian preconditioning and Latent Binary ADMM (LB-ADMM) initialization used in NANOQUANT. The LB-ADMM objective is nonconvex and includes discrete proxy updates; therefore, we do not claim global convergence or monotonic descent of the augmented Lagrangian. Instead, we show that the preconditioned target remains spectrally controlled and that the continuous ADMM subproblems are well-posed symmetric positive definite linear systems. These properties justify LB-ADMM as a stable initialization procedure for the subsequent block reconstruction stage.

B.1. Problem Setup

Let $\mathbf{W}_{\text{FP}} \in \mathbb{R}^{m \times n}$ denote a full-precision weight matrix. We define the robustly preconditioned target as

$$\mathbf{W} \triangleq \tilde{\mathbf{D}}_{\text{out}} \mathbf{W}_{\text{FP}} \tilde{\mathbf{D}}_{\text{in}} \in \mathbb{R}^{m \times n}, \quad (26)$$

where $\tilde{\mathbf{D}}_{\text{out}} \in \mathbb{R}^{m \times m}$ and $\tilde{\mathbf{D}}_{\text{in}} \in \mathbb{R}^{n \times n}$ are diagonal preconditioners constructed from robust calibration statistics.

LB-ADMM seeks a rank- R factorization $\mathbf{W} \approx \mathbf{U}\mathbf{V}^\top$ with $\mathbf{U} \in \mathbb{R}^{m \times R}$ and $\mathbf{V} \in \mathbb{R}^{n \times R}$. To separate continuous reconstruction from the discrete proxy structure, we introduce auxiliary variables $\mathbf{Z}_{\mathbf{U}}$ and $\mathbf{Z}_{\mathbf{V}}$. The regularized constrained objective is

$$\begin{aligned} \min_{\mathbf{U}, \mathbf{V}, \mathbf{Z}_{\mathbf{U}}, \mathbf{Z}_{\mathbf{V}}} \quad & \frac{1}{2} \|\mathbf{W} - \mathbf{U}\mathbf{V}^\top\|_F^2 + \frac{\lambda}{2} (\|\mathbf{U}\|_F^2 + \|\mathbf{V}\|_F^2) + \mathcal{I}_{\mathcal{C}_{\mathbf{U}}}(\mathbf{Z}_{\mathbf{U}}) + \mathcal{I}_{\mathcal{C}_{\mathbf{V}}}(\mathbf{Z}_{\mathbf{V}}) \\ \text{s.t.} \quad & \mathbf{U} = \mathbf{Z}_{\mathbf{U}}, \quad \mathbf{V} = \mathbf{Z}_{\mathbf{V}}, \end{aligned} \quad (27)$$

where $\lambda \geq 0$ is the ridge regularization coefficient, and $\mathcal{I}_{\mathcal{C}_{\mathbf{U}}}$ and $\mathcal{I}_{\mathcal{C}_{\mathbf{V}}}$ are indicator functions for the structured proxy families used to initialize the latent binary factors.

B.2. Spectral Control from Robust Preconditioning

The stability of the initialization depends partly on controlling the scale of the preconditioned target $\mathbf{W}$. The robust diagonal estimator used in NANOQUANT applies clipping and shrinkage to reduce the influence of outlier calibration statistics.

Lemma 1 (Bounded preconditioners). *Assume that the diagonal preconditioner entries are nonnegative and clipped to a maximum value $\tau_{\max} > 0$. Then the diagonal preconditioners satisfy*

$$\|\tilde{\mathbf{D}}_{\text{out}}\|_2 \leq \tau_{\max}, \quad \|\tilde{\mathbf{D}}_{\text{in}}\|_2 \leq \tau_{\max}. \quad (28)$$

Proof. For a diagonal matrix, the spectral norm is the maximum absolute diagonal entry. Since each diagonal entry of $\tilde{\mathbf{D}}_{\text{out}}$ and $\tilde{\mathbf{D}}_{\text{in}}$ is nonnegative and clipped to be at most $\tau_{\max}$, their spectral norms are bounded by $\tau_{\max}$. $\square$

Corollary 1 (Bounded preconditioned target). *The preconditioned target satisfies*

$$\|\mathbf{W}\|_2 \leq \tau_{\max}^2 \|\mathbf{W}_{\text{FP}}\|_2. \quad (29)$$

Proof. Using submultiplicativity of the spectral norm,

$$\begin{aligned} \|\mathbf{W}\|_2 &= \left\| \tilde{\mathbf{D}}_{\text{out}} \mathbf{W}_{\text{FP}} \tilde{\mathbf{D}}_{\text{in}} \right\|_2 \\ &\leq \|\tilde{\mathbf{D}}_{\text{out}}\|_2 \|\mathbf{W}_{\text{FP}}\|_2 \|\tilde{\mathbf{D}}_{\text{in}}\|_2 \leq \tau_{\max}^2 \|\mathbf{W}_{\text{FP}}\|_2. \end{aligned} \quad (30)$$

$\square$

This bound does not imply global convergence of the nonconvex factorization problem. However, it ensures that the target matrix passed to LB-ADMM has controlled spectral scale, which helps avoid extreme numerical ranges in the subsequent linear solves.

B.3. LB-ADMM Updates

We use the scaled-dual ADMM form with penalty parameter $\rho > 0$ and scaled dual variables $\Lambda_{\mathbf{U}}$ and $\Lambda_{\mathbf{V}}$. Given $\mathbf{V}$, $\mathbf{Z}_{\mathbf{U}}$, and $\Lambda_{\mathbf{U}}$, the continuous update for $\mathbf{U}$ solves

$$\min_{\mathbf{U}} \quad \frac{1}{2} \|\mathbf{W} - \mathbf{U}\mathbf{V}^\top\|_F^2 + \frac{\lambda}{2} \|\mathbf{U}\|_F^2 + \frac{\rho}{2} \|\mathbf{U} - \mathbf{Z}_{\mathbf{U}} + \Lambda_{\mathbf{U}}\|_F^2. \quad (31)$$

The corresponding normal equation is

$$\mathbf{U} (\mathbf{V}^\top \mathbf{V} + (\rho + \lambda) \mathbf{I}) = \mathbf{W}\mathbf{V} + \rho (\mathbf{Z}_{\mathbf{U}} - \Lambda_{\mathbf{U}}). \quad (32)$$

Equivalently,

$$(\mathbf{V}^\top \mathbf{V} + (\rho + \lambda) \mathbf{I}) \mathbf{U}^\top = \mathbf{V}^\top \mathbf{W}^\top + \rho (\mathbf{Z}_{\mathbf{U}} - \Lambda_{\mathbf{U}})^\top. \quad (33)$$

The update for $\mathbf{V}$ is symmetric:

$$\mathbf{V} (\mathbf{U}^\top \mathbf{U} + (\rho + \lambda) \mathbf{I}) = \mathbf{W}^\top \mathbf{U} + \rho (\mathbf{Z}_{\mathbf{V}} - \Lambda_{\mathbf{V}}). \quad (34)$$

The proxy variables are updated using the SVID operator:

$$\mathbf{Z}_{\mathbf{U}} \leftarrow \text{SVID}(\mathbf{U} + \Lambda_{\mathbf{U}}), \quad \mathbf{Z}_{\mathbf{V}} \leftarrow \text{SVID}(\mathbf{V} + \Lambda_{\mathbf{V}}). \quad (35)$$

This step imposes the structured sign-value proxy used for latent binary initialization. Since the proxy families are nonconvex and discrete, the SVID step should be interpreted as a structured proxy update rather than a convex projection.

Finally, the scaled dual variables are updated as

$$\Lambda_{\mathbf{U}} \leftarrow \Lambda_{\mathbf{U}} + \mathbf{U} - \mathbf{Z}_{\mathbf{U}}, \quad \Lambda_{\mathbf{V}} \leftarrow \Lambda_{\mathbf{V}} + \mathbf{V} - \mathbf{Z}_{\mathbf{V}}. \quad (36)$$

B.4. Well-Posed Continuous Updates

Lemma 2 (Positive definiteness of the continuous subproblems). *For any $\rho > 0$ and $\lambda \geq 0$, the system matrices*

$$\mathbf{H}_\mathbf{U} = \mathbf{V}^\top \mathbf{V} + (\rho + \lambda)\mathbf{I}, \quad \mathbf{H}_\mathbf{V} = \mathbf{U}^\top \mathbf{U} + (\rho + \lambda)\mathbf{I} \quad (37)$$

are symmetric positive definite. Consequently, the continuous updates for $\mathbf{U}$ and $\mathbf{V}$ are strongly convex quadratic subproblems with unique solutions.

Proof. For any nonzero vector $\mathbf{a}$,

$$\mathbf{a}^\top (\mathbf{V}^\top \mathbf{V} + (\rho + \lambda)\mathbf{I}) \mathbf{a} = \|\mathbf{V}\mathbf{a}\|_2^2 + (\rho + \lambda)\|\mathbf{a}\|_2^2. \quad (38)$$

Since $\rho > 0$ and $\lambda \geq 0$, this quantity is strictly positive. Thus, $\mathbf{H}_\mathbf{U}$ is positive definite. The same argument applies to $\mathbf{H}_\mathbf{V}$. The corresponding objectives are therefore strongly convex quadratics in the updated variable and admit unique minimizers. $\square$

Corollary 2 (Conditioning of the linear solves). *The condition number of the $\mathbf{U}$ update matrix satisfies*

$$\kappa(\mathbf{H}_\mathbf{U}) = \frac{\sigma_{\max}(\mathbf{V})^2 + \rho + \lambda}{\sigma_{\min}(\mathbf{V})^2 + \rho + \lambda} \leq 1 + \frac{\|\mathbf{V}\|_2^2}{\rho + \lambda}. \quad (39)$$

An analogous bound holds for the $\mathbf{V}$ update matrix:

$$\kappa(\mathbf{H}_\mathbf{V}) \leq 1 + \frac{\|\mathbf{U}\|_2^2}{\rho + \lambda}. \quad (40)$$

Proof. The eigenvalues of $\mathbf{V}^\top \mathbf{V}$ are the squared singular values of $\mathbf{V}$. Adding $(\rho + \lambda)\mathbf{I}$ shifts every eigenvalue by $\rho + \lambda$, giving

$$\kappa(\mathbf{H}_\mathbf{U}) = \frac{\sigma_{\max}(\mathbf{V})^2 + \rho + \lambda}{\sigma_{\min}(\mathbf{V})^2 + \rho + \lambda}. \quad (41)$$

Since $\sigma_{\min}(\mathbf{V})^2 \geq 0$, we have

$$\kappa(\mathbf{H}_\mathbf{U}) \leq \frac{\sigma_{\max}(\mathbf{V})^2 + \rho + \lambda}{\rho + \lambda} = 1 + \frac{\|\mathbf{V}\|_2^2}{\rho + \lambda}. \quad (42)$$

The proof for $\mathbf{H}_\mathbf{V}$ is identical. $\square$

This result explains the roles of the ADMM penalty and ridge regularization. The diagonal shift $(\rho + \lambda)\mathbf{I}$ guarantees positive definiteness and prevents singular continuous updates, even when one factor is rank-deficient. Increasing ρ strengthens the coupling between the continuous factors and the structured proxy variables, while increasing λ further controls the factor magnitudes. Both terms improve the worst-case conditioning of the Cholesky solves through the same positive diagonal shift.

B.5. Role of the SVID Proxy Step

The SVID update imposes the sign-value structure required for latent binary initialization. Because the proxy families $\mathcal{C}_\mathbf{U}$ and $\mathcal{C}_\mathbf{V}$ are nonconvex and discrete, standard convex ADMM convergence guarantees do not directly apply. Accordingly, we use LB-ADMM as a stable initialization procedure rather than as a globally convergent solver for the discrete factorization problem.

The optimization remains well-posed in the sense required for initialization: each continuous update is a unique SPD solve, and each proxy update maps the continuous factor back to the structured family used to initialize binary factors. The subsequent magnitude balancing and block reconstruction stages further refine these initialized factors using calibration data.

Summary. The above analysis establishes three stability properties of LB-ADMM initialization. First, robust preconditioning bounds the spectral scale of the target matrix. Second, the continuous ADMM updates are well-posed SPD linear systems with unique solutions. Third, the penalty and ridge terms help control the conditioning of these solves. These properties do not constitute a global convergence proof for the nonconvex discrete problem, but they justify LB-ADMM as a stable and computationally efficient initialization procedure for low-rank binary quantization.

C. Implementation Details

All compression experiments for NANOQUANT were conducted on 1 NVIDIA H100 (80 GB), and we utilize unified hyper-parameters when compressing models with NANOQUANT. When tuning pre-factorized parameters to absorb quantization error, we used a learning rate of $1e-4$ and a batch size of 4. For tuning factorized parameters (low-rank, latent binary and full-precision scales), we used a unified learning rate of $1e-5$ and a batch size of 1. For global scale reconstruction, we used a learning rate of $1e-6$ and a batch size of 1. Pre-factorized, factorized, and global tuning stages all consist of 8 epochs and utilize a cosine learning rate scheduler. We employed a linear ADMM penalty scheduler for 400 factorization steps, for each weight matrix across all models. For calibration data, we used 128 samples with a sequence length of 2048 from the WikiText-2 dataset (Merity et al., 2016), and used a random seed value of 0 for data selection. For all experiments, we used `torch=2.6.0`, `transformers=4.51.3`, `datasets=4.0.0`, `lm_eval=0.4.9`, and CUDA 12.4. To derive the activation- and gradient-based diagonal preconditioners, we utilized gradient checkpointing and used a memory-efficient implementation of the cross-entropy function (Wijmans et al., 2025). During block reconstruction, we employed a weighted MSE function, utilized in previous quantization works (Boža & Macko, 2026; Kim et al., 2025). Official open-source implementations or quantized models were used to evaluate baselines for binary PTQ (Huang et al., 2024; Dong et al., 2025; Li et al., 2025; Chen et al., 2026), vector quantization (Egiazarian et al., 2024; Malinovskii et al., 2024; Tseng et al., 2024b), and low-rank binary QAT (Boža & Macko, 2026; Lee et al., 2025a).

Since none of the binary PTQ baselines (BiLLM (Huang et al., 2024), STBLLM (Dong et al., 2025), ARB-LLM (Li et al., 2025), HBLLM (Chen et al., 2026)) compress quantized models into memory-efficient formats, we utilize the main text and appendices of such methods to calculate both effective bits and model checkpoint sizes (Huang et al., 2024; Li et al., 2025; Chen et al., 2026). Further details on effective bit calculation and model checkpoint sizes can be found in Appendix F.

D. Further Ablations

D.1. Different Data Budgets

We test with different calibration data budgets for the block and model reconstruction stages. As in Table 9, utilizing more data during block reconstruction leads to greater performance gains.

Table 9. Utilizing different data budgets for the block and model reconstruction stages of NANOQUANT, when compressing Llama-2-7B to 1-bit.

Block Recon. Samples	Model Recon. Samples				
	32	64	128	256	512
32	14.16	13.24	12.72	12.22	11.91
64	12.13	11.66	11.23	10.94	10.69
128	10.56	10.40	10.35	10.06	9.89
256	10.25	10.24	10.17	9.77	9.60
512	9.24	9.16	9.13	9.07	9.07

D.2. Different Calibration Datasets

We test how the calibration dataset impacts performance. Table 10 shows that increasing calibration data diversity mitigates overfitting and can improve zero-shot performance.

Table 10. Effect of calibration dataset composition on 1-bit quantization for instruction-tuned Qwen3-4B and Llama-2-7B. We vary the number of calibration samples drawn from C4 and WikiText-2 (WT2), while keeping the total fixed at 128.

Samples		Qwen3-4B, 1-bit			Llama-2-7B, 1-bit		
C4	WT2	WT2 PPL ($\downarrow$)	C4 PPL ($\downarrow$)	Zero-shot ($\uparrow$)	WT2 PPL ($\downarrow$)	C4 PPL ($\downarrow$)	Zero-shot ($\uparrow$)
0	128	21.25	58.28	46.53	10.34	23.23	48.68
32	96	22.11	41.24	48.66	10.61	17.92	49.06
64	64	22.66	39.22	47.08	10.74	17.15	50.08
96	32	23.92	36.41	50.19	11.42	16.25	50.87
128	0	38.96	35.23	50.22	17.99	16.31	48.70

D.3. Analysis of Latent Weight Dynamics

To validate the efficacy of the Factorized Component Refinement phase (Step 3), we analyze the trajectory of the continuous latent variables, $\mathcal{U}$ and $\mathcal{V}$, before and after fine-tuning. Figure 8 visualizes the distribution shifts and sign flip ratios for all linear layers within the first transformer block of Llama-3.2-1B. The abscissa represents the magnitude of the latent weights initialized by LB-ADMM (Step 2), while the ordinate in the right-hand panels denotes the magnitude of change after refinement (Step 3).

Stability of Initialization. As illustrated in Figure 8, a predominant proportion of latent weights retain their original signs throughout the refinement process. The sign flip ratio remains consistently low, ranging from 0.47% in the `k_proj` layer to 6.82% in the `gate_proj` layer. This stability indicates that the LB-ADMM initialization establishes a parameter configuration proximate to a local optimum, thereby mitigating the necessity for substantial updates during the fine-tuning phase.

Refinement of Boundary Weights. Despite the low flip ratio, the refinement step functions as a critical margin maximization process. The interaction density plots reveal an inverse correlation between the initial magnitude and the degree of change. Specifically, weights with initial magnitudes near zero, corresponding to the decision boundary, exhibit the highest mobility and likelihood of sign flipping. This behavior suggests that the refinement phase selectively rectifies the signs of “ambiguous” weights near the zero boundary while preserving the confident decisions established during the initialization. Consequently, this targeted adjustment compensates for discretization errors introduced during the initial factorization.

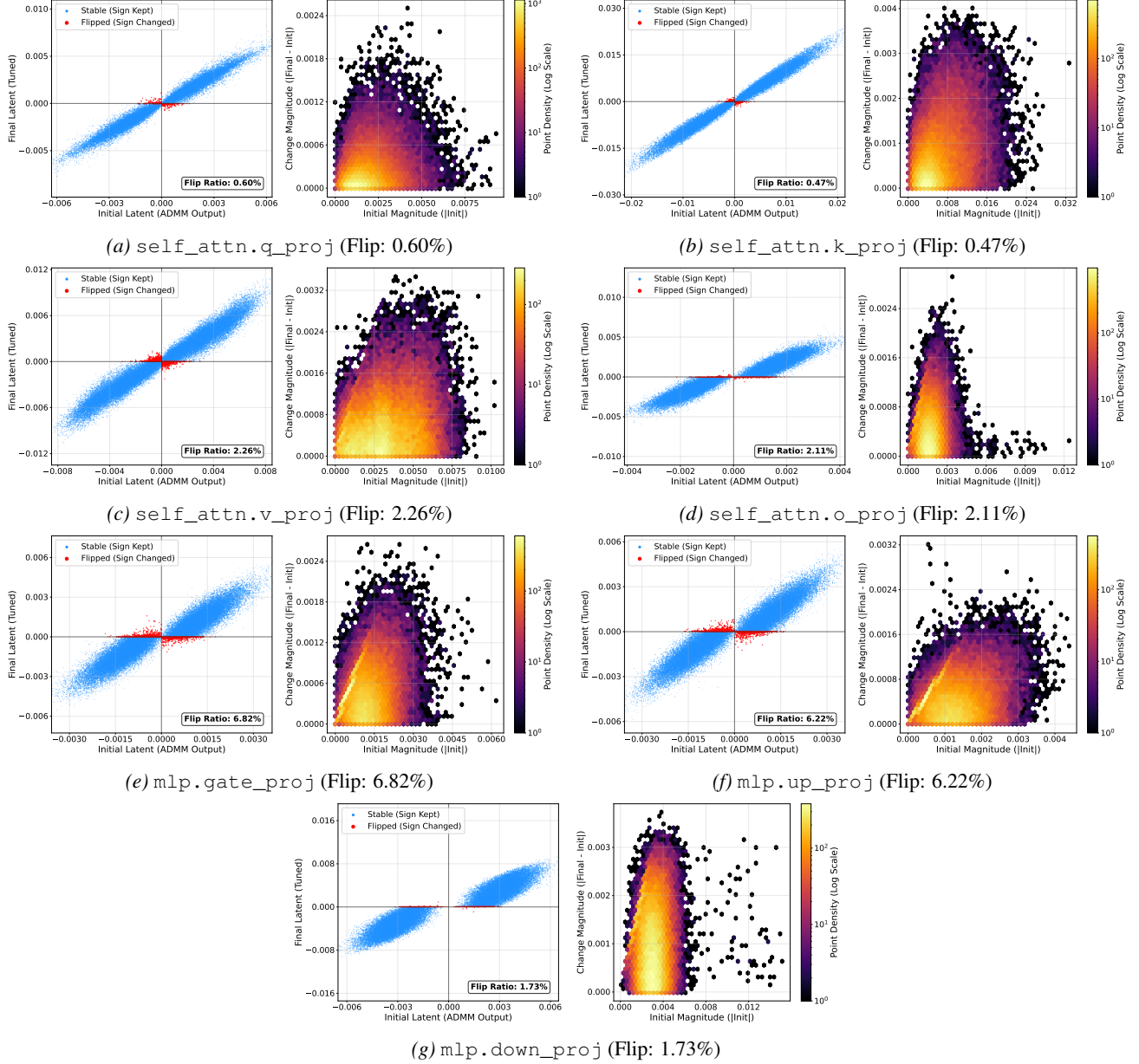
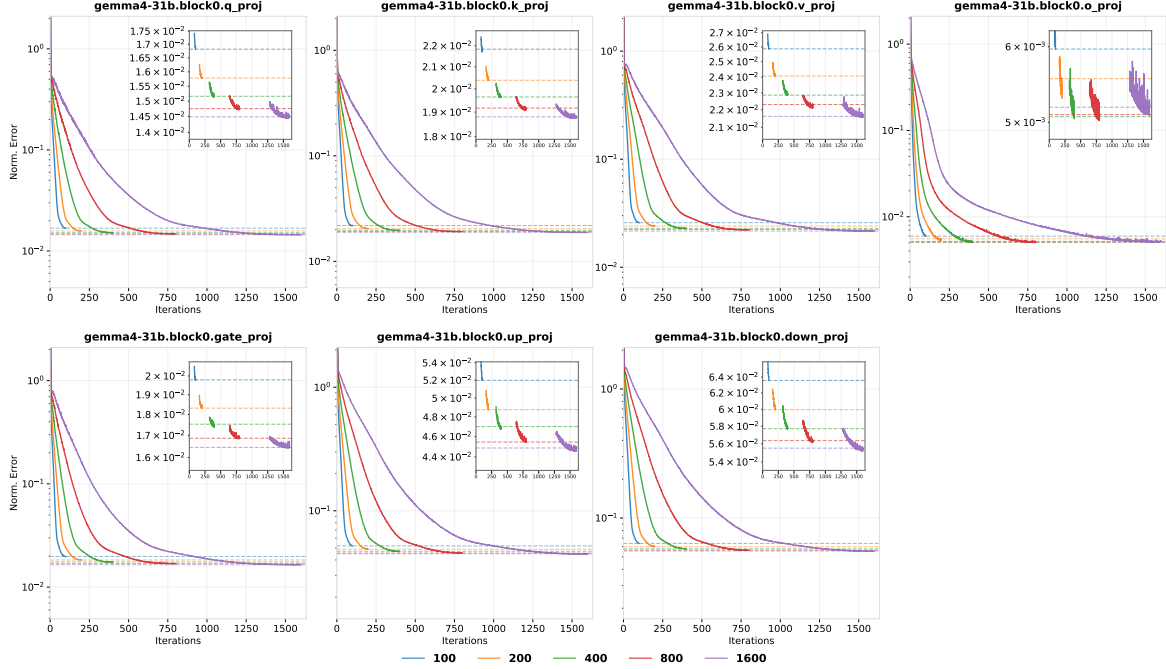


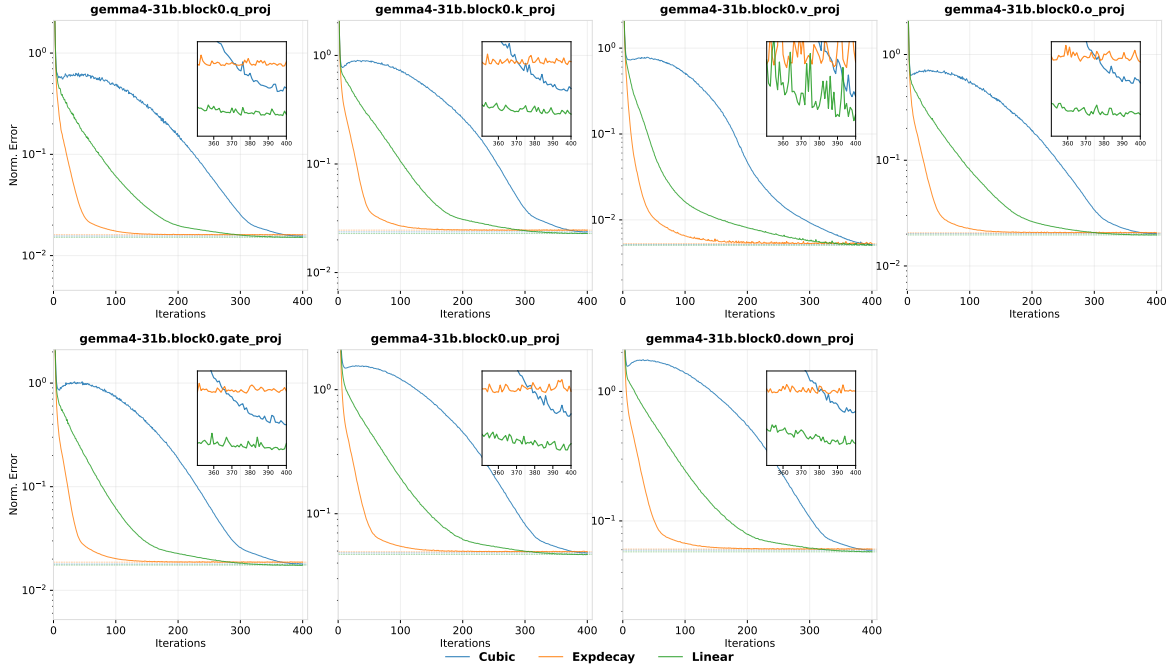
Figure 8. Visualization of latent variable dynamics between the initialization (LB-ADMM) and refinement (STE Tuning) phases for Llama-3.2-1B (Block 0). Blue points denote weights that retained their sign, while red points denote sign flips. The density plots in each panel illustrate that sign flips and large magnitude updates are concentrated around weights with near-zero initial magnitude. This demonstrates that the refinement step selectively optimizes decision boundaries for features with high uncertainty.

D.4. ADMM Ablation Experiments

We ablate two ADMM optimization choices on block 0 of Gemma 4 31B: the number of outer iterations and the penalty scheduling strategy. The results show a clear speed-accuracy trade-off. Reducing the number of ADMM iterations gives faster convergence but leads to a higher final reconstruction error.



(a) Effect of ADMM outer iterations. Fewer iterations reduce optimization cost but produce higher final reconstruction error.



(b) Effect of ADMM penalty scheduling. Linear scheduling converges more slowly but achieves lower final error than more aggressive schedules.

Figure 9. ADMM ablations on Gemma-4-31B block 0. The number of outer iterations controls the optimization cost, while the penalty scheduler controls the convergence profile and final reconstruction quality.

E. Inference Ablations

E.1. Kernel Benchmarking Details

We benchmark custom CUDA kernels using `torch.compile` from torch 2.6.0 (Paszke et al., 2019) and `StaticCache` from the transformers library (Wolf et al., 2020), with CUDA 12.4. The decoding script is based on the open-source implementation for QTIP (Tseng et al., 2024b), and is used for all kernel evaluations. For GEMV decoding (batch size = 1), we vary the number of output tokens. For GEMM inference, we evaluate performance under increasing batch sizes. We fix the input tokens to 128, output tokens for batched inference to 512, temperature to 0.8, and the top-k value to 32. We utilize the open-source `ml-energy/zeus` library for all energy measurements.

We benchmark our kernels on 4 different GPUs, as listed in Table 11. Notably, we test on high-end GPUs, a consumer GPU, and an edge device with *no NVIDIA Tensor Cores*, to test the decoding and efficiency limits of our custom binary CUDA kernels.

Table 11. Hardware specifications of devices we benchmark our custom GPU kernels on.

Device	Memory			Compute	
	GPU Memory (GB)	Type	Bandwidth (GB/s)	CUDA Cores	Tensor Cores
NVIDIA Jetson TX2	8	LPDDR4	59.7	256	0
NVIDIA RTX 3050	8	GDDR6	224	2,560	80
NVIDIA A100 SXM	80	HBM2e	2039	6,912	432
NVIDIA H100 PCIe	80	HBM2e	2000	14,592	456

E.2. Binary GEMV Inference

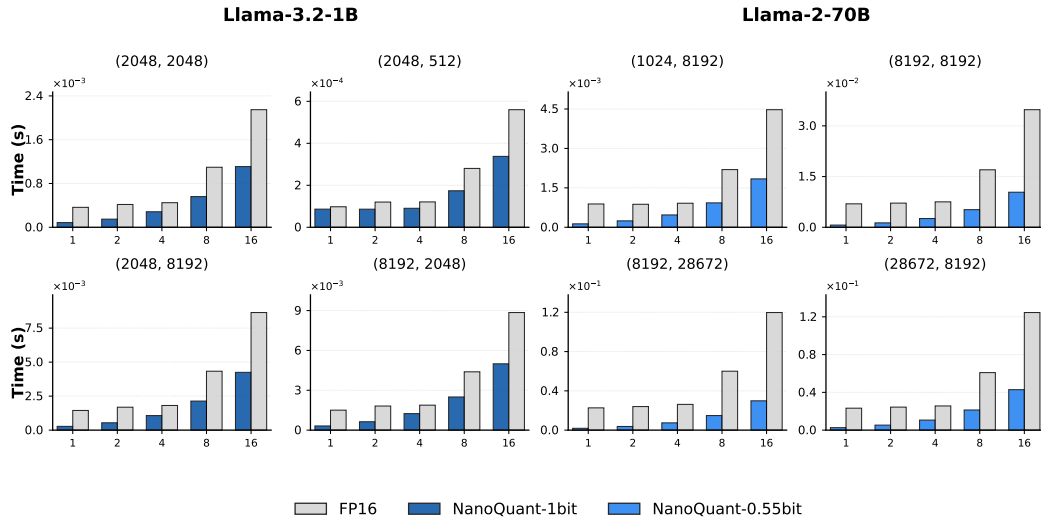


Figure 10. On the NVIDIA Jetson TX2, our custom GEMV kernels show significantly faster inference speeds than PyTorch FP16 for various matrix shapes, even for multiple vector batches.

GEMV CUDA kernel details. The GEMV kernel implements a two-stage, 1-bit quantized matrix–vector multiplication for `float16` and `bfloat16` tensors. In each stage, the input (or intermediate) vector is multiplied by a weight matrix whose signs are stored as a packed bitfield: each weight occupies a single bit in a `uint32` array. Because the binary matrices are low-rank, the effective reduction in memory traffic is typically less than the theoretical 16×, but the packing still yields a substantial bandwidth saving. During execution the bits are unpacked on-the-fly with a lightweight mask operation, after which fused-multiply-add (FMA) is performed using vectorized `float16` or `bfloat16` intrinsics that process two low-precision values per instruction. Per-column scaling factors are incorporated directly into the FMA, and an optional per-row scaling is applied before writing the intermediate or final result. The kernel does not invoke Tensor-Core WMMA

Table 12. Throughput (tokens/s) and peak memory (GB) for varying sequence lengths, for Llama-2 models compressed to 0.55 bits with NANOQUANT. Extreme compression with NANOQUANT enables fast and memory-efficient inference on an NVIDIA RTX 3050 (8 GB).

Model	Metric	Sequence Length					
		32	64	128	256	512	1024
Llama-2-7B	Tokens/s	134.10	133.40	127.04	122.52	108.44	86.27
	Peak Mem (GB)	1.07	1.09	1.12	1.19	1.32	1.59
Llama-2-13B	Tokens/s	83.83	83.35	81.43	75.32	65.31	51.63
	Peak Mem (GB)	1.70	1.73	1.78	1.88	2.09	2.57
Llama-2-70B	Tokens/s	20.11	19.74	19.18	17.68	15.37	12.13
	Peak Mem (GB)	5.86	5.87	5.89	5.93	6.02	6.20

operations; instead it relies on standard FP16 arithmetic, without using NVIDIA Tensor Cores.

Results. We first benchmark our binary GEMV CUDA kernels against PyTorch BF16 and 2 state-of-the-art vector quantization methods, QTIP (Tseng et al., 2024b) and AQLM (Egiazarian et al., 2024), on a single NVIDIA H100 GPU, as shown in Figure 7. To fully encompass the performance of the kernels, we measure the throughput (tokens per second), peak allocated GPU memory, and average energy per token during the single batch, end-to-end decoding process. We utilize open-source models provided by QTIP and AQLM, and utilize models from the Llama-2 (Touvron et al., 2023) and Llama-3 (Grattafiori et al., 2024) families.

Next, we benchmark our binary GEMV CUDA kernels on an NVIDIA Jetson TX2, which does not have any NVIDIA Tensor Cores, as in Table 11. We find that the extreme compression capability of NANOQUANT enables up to a $12.2\times$ speedup in inference throughput, compared to PyTorch FP16, as shown in Figure 10.

E.3. Binary GEMM Inference

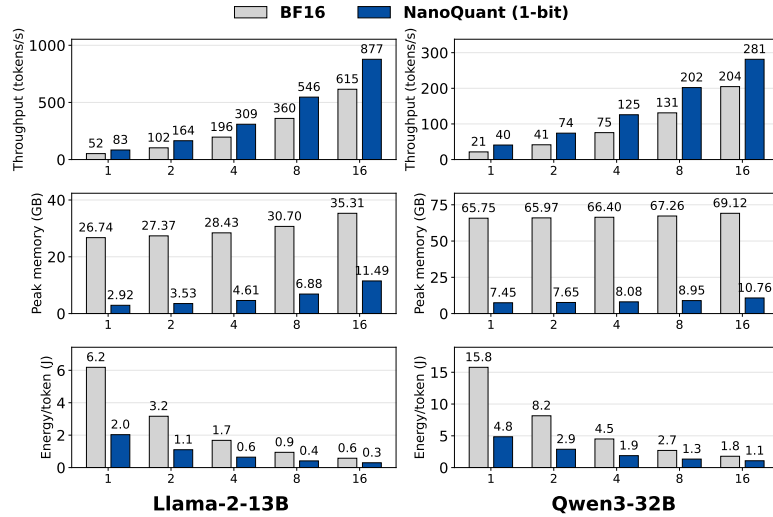


Figure 11. Custom GEMM kernels for NANOQUANT achieve competitive batched inference performance with BF16 PyTorch on a single NVIDIA A100 (80 GB) GPU.

GEMM Kernel Details Our GEMM kernel is a highly optimized CUDA GEMM implementation specifically designed for efficient low-rank binary matrix multiplication in quantized neural networks. We base our implementation on the Marlin GEMM kernel (Frantar et al., 2025), which leverages NVIDIA Tensor Cores for matrix multiplication through inline PTX assembly, processing matrix (e.g., $16 \times 8 \times 16$) tiles with `mma.sync` operations. The kernel employs a multi-stage

pipeline (default 4 stages) with asynchronous memory operations (`cp.async`) to overlap data transfers with computation, effectively hiding memory latency. It efficiently handles binary matrices by packing multiple 1-bit values into 32-bit words and using bit manipulation for fast dequantization, while maintaining computation in FP16/BF16 precision for accuracy.

While GEMV excels in low-batch, memory-bound scenarios, large-scale deployments benefit from batched GEMM operations that saturate tensor core throughput. Therefore, Binary GEMM kernels are necessary for compute-bound LLM serving operations, especially for datacenter GPUs to fully utilize matrix-multiplication computation units, such as NVIDIA Tensor Cores. The pipelined execution keeps compute units busy by overlapping the data loading, computation, and storing phases, while the warp-level parallelism with optimized thread scheduling maximizes GPU utilization. These optimizations result in high arithmetic intensity and efficient use of hardware resources, making the kernel particularly well-suited for quantized neural network inference where binary low-rank matrices significantly decrease memory requirements and bandwidth usage without sacrificing model accuracy.

E.4. Binary GPU Kernel Performance

We compare the decoding performance of our binary GPU kernels with GemLite (Hicham Badri, 2024), a state-of-the-art GPU kernel library that supports 1-bit kernels. As shown in Figure 12 and Figure 13, our custom binary GEMV kernels outperform both BF16 and GemLite, with noNVIDIA Tensor Core free decoding operations.

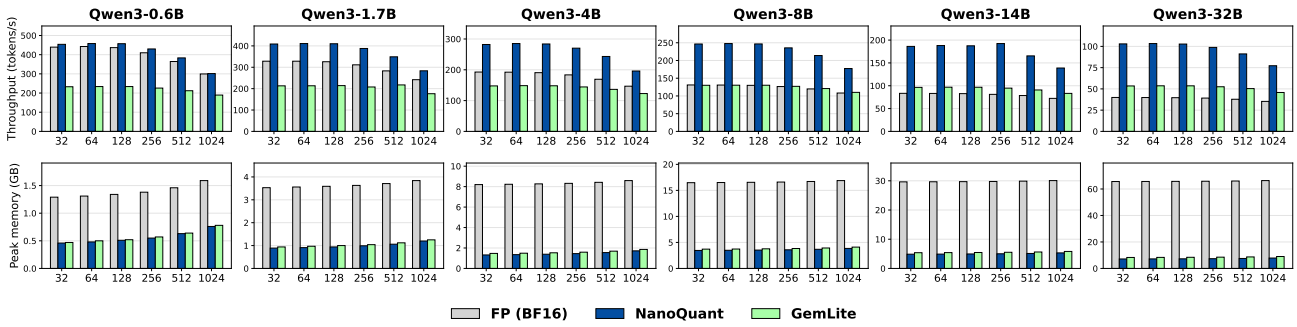


Figure 12. LLM decoding performance of NANOQUANT using our custom binary kernels and binary kernels from GemLite (Hicham Badri, 2024), compared with PyTorch BF16 on 1 NVIDIA H100 GPU.

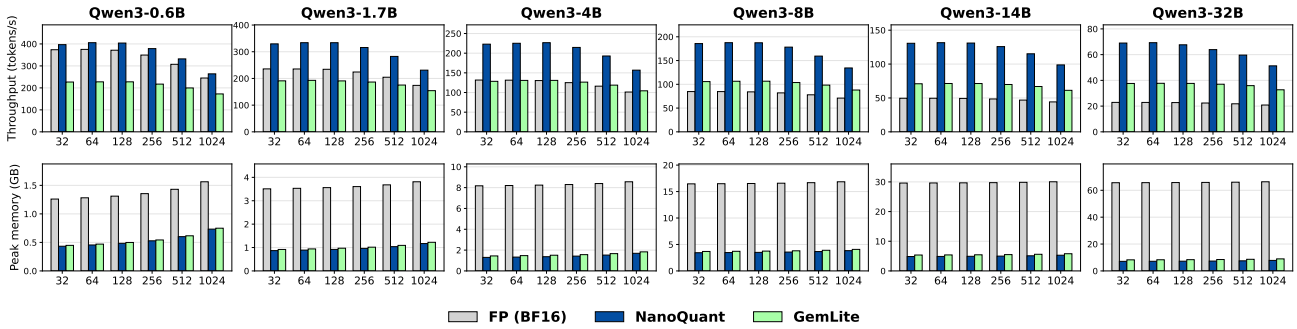


Figure 13. LLM decoding performance of NANOQUANT using our custom binary kernels and binary kernels from GemLite (Hicham Badri, 2024), compared with PyTorch BF16 on 1 NVIDIA A100 GPU.

F. Detailed Model Size Analysis of Binary Weight Quantization Methods

We analyze the storage cost of binary weight representation methods, encompassing both post-training quantization (PTQ) and quantization-aware training (QAT) via low-rank factorization. We count all stored bits including binarized weights (or binary factor matrices), reconstruction coefficients (typically FP16), and any auxiliary flags or bitmaps required to accurately decode the quantized model. This allows for a fair assessment of the memory requirements across diverse binary compression paradigms.

F.1. Unified Storage Metric

Bits Per Weight (BPW). Let B_{W_Q} and $B_{W_{FP}}$ be the total number of bits required to represent quantized weights and full-precision weights, respectively. We derive BPW, the average number of bits per full-precision weight in the quantized model, as:

$$\text{BPW} = \frac{B_{W_Q}}{B_{W_{FP}}} \times 16. \quad (43)$$

For instance, if we binarize all weight values in a full-precision matrix $\mathbf{W}_{FP} \in \mathbb{R}^{n \times m}$ to ± 1 without additional metadata, the BPW value would be $\frac{nm}{16nm} \times 16 = 1$.

If we assume $\mathbf{W}_Q$ is a two-dimensional matrix, $B_{W_{FP}} = 16mn$, and thus $\text{BPW} = \frac{B_{W_Q}}{16mn} \times 16 = \frac{B_{W_Q}}{mn}$.

F.2. Overview of Memory Requirements of Binary Quantization Methods

Notation. We consider a weight matrix $\mathbf{W} \in \mathbb{R}^{n \times m}$, where n is the number of rows and m is the number of columns. We define k as the block size (typically 128), and c as the number of salient columns. The number of blocks per row is denoted as $\lceil m/k \rceil$. We assume reconstruction scales and means are stored in FP16 (16 bits). Additionally, m denotes the storage cost for the salient column bitmap (often compressed).

Methods. We analyze the memory requirements of state-of-the-art binary PTQ methods (BiLLM (Huang et al., 2024), ARB-LLM (Li et al., 2025), STBLLM (Dong et al., 2025), HBLLM (Chen et al., 2026)), binary QAT methods using low-rank binary matrices (DBF (Boža & Macko, 2026), LittleBit (Lee et al., 2025a)), and NANOQUANT.

F.3. Binary PTQ Methods

BiLLM. BiLLM (Huang et al., 2024) partitions the weight matrix into salient and non-salient parts. It employs second-order binarization for salient columns and first-order binarization with two quantization groups for non-salient columns. Based on the analysis in (Chen et al., 2026), the total memory requirement $\mathcal{M}_{\text{BiLLM}}$ is formulated as:

$$\begin{aligned} \mathcal{M}_{\text{BiLLM}} &= \underbrace{2nc + \lceil m/k \rceil \times 3n \times 16}_{\text{Second-order binarization (Salient)}} \\ &\quad + \underbrace{n(m - c) + \lceil m/k \rceil \times 2n \times 16 \times 2}_{\text{First-order binarization (Non-salient, 2 groups)}} \\ &\quad + \underbrace{nm}_{\text{Non-salient group bitmap}} + \underbrace{m}_{\text{Salient column bitmap}} \\ &= n(2m + c) + m + 112n \lceil m/k \rceil, \end{aligned} \quad (44)$$

where c is the number of salient columns. The term $3n$ in the second-order part accounts for parameters α_1, α_2 and the combined mean μ .

With $\mathcal{M}_{\text{BiLLM}}$, we can derive the BPW equation as

$$\text{BPW}_{\text{BiLLM}} = \frac{\mathcal{M}_{\text{BiLLM}}}{mn} = 2 + \frac{nc + m + 112n \lceil m/k \rceil}{mn}. \quad (45)$$

STBLLM STBLLM (Dong et al., 2025) extends the BiLLM framework by introducing $N : M$ sparsity and finer-grained grouping. Unlike BiLLM, which uses 2 groups, STBLLM categorizes non-salient weights into 3 groups (sparse, intermediate, dense) using a trisection search, requiring a 2-bit group bitmap per stored element.

Additionally, STBLLM employs $N : M$ structured sparsity (e.g., 4:8 or 6:8) for the non-salient weights. This requires storing the indices of the non-zero elements. For a standard $N : M$ pattern, the index storage $\mathcal{M}_{\text{Indices}}$ is determined by the combinatorics of choosing N positions out of M , typically $\lceil \log_2 \binom{M}{N} \rceil$ bits per block of M weights.

The total memory requirement $\mathcal{M}_{\text{STBLLM}}$ is formulated as:

$$\begin{aligned}
 \mathcal{M}_{\text{STBLLM}} = & \underbrace{2nc + \lceil m/k \rceil \cdot 3n \cdot 16}_{\text{Second-order binarization (Salient)}} \\
 & + \underbrace{\frac{N}{M} [n(m-c) + 2nm]}_{\text{Binarized non-zero weights and Group bitmap}} \\
 & + \underbrace{\frac{n(m-c)}{M} \cdot \left\lceil \log_2 \binom{M}{N} \right\rceil}_{\text{Sparsity Indices (Metadata)}} \\
 & + \underbrace{\lceil m/k \rceil \cdot 2n \cdot 16 \cdot 3}_{\text{FP16 scales/means (3 groups)}} + \underbrace{m}_{\text{Salient column bitmap}}
 \end{aligned} \tag{46}$$

where n and m are the matrix dimensions, c is the number of salient columns, and k is the block size. Dividing by the total original parameters mn yields the Bit-Width Per Weight (BPW) equation:

$$\begin{aligned}
 \text{BPW}_{\text{STBLLM}} &= \frac{\mathcal{M}_{\text{STBLLM}}}{mn} \\
 &= \frac{N}{M} \left(1 - \frac{c}{m} + 2 \right) + \frac{2c}{m} + \frac{1}{M} \left(1 - \frac{c}{m} \right) \left\lceil \log_2 \binom{M}{N} \right\rceil \\
 &+ \frac{144n \lceil m/k \rceil + m}{mn}
 \end{aligned} \tag{47}$$

ARB-LLM ARB-LLM (Li et al., 2025) utilizes alternating refined binarization. We analyze the storage for the ARB-LLM_{RC} variant, as derived in (Li et al., 2025). This method applies second-order binarization to both salient and non-salient parts using 2 groups:

$$\begin{aligned}
 \mathcal{M}_{\text{ARBLLM-RC}} &= \underbrace{2nc + (\lceil m/k \rceil \times 2n + 2c) \times 16}_{\text{Second-order binarization (Salient, 2 groups)}} \\
 &+ \underbrace{n(m-c) + (\lceil m/k \rceil \times n + (m-c)) \times 16 \times 2}_{\text{First-order binarization (Non-salient, 2 groups)}} \\
 &+ \underbrace{nm}_{\text{Group bitmap}} + \underbrace{m}_{\text{Salient column bitmap}} \\
 &= n(2m + c) + 33m + 64n \lceil m/k \rceil.
 \end{aligned} \tag{48}$$

With $\mathcal{M}_{\text{ARBLLM-RC}}$, we can derive the BPW equation as

$$\text{BPW}_{\text{ARBLLM-RC}} = \frac{\mathcal{M}_{\text{ARBLLM-RC}}}{mn} = 2 + \frac{nc + 33m + 64n \lceil m/k \rceil}{mn}. \tag{49}$$

HBLLM HBLLM (Chen et al., 2026) introduces structure-aware grouping with two primary variants: HBLLM-row and HBLLM-col.

HBLLM-row employs a neighborhood averaging strategy for non-salient weights and utilizes four subgroups per row for coefficients:

$$\begin{aligned}
 \mathcal{M}_{\text{HBLLM-row}} &= \underbrace{nm + \lceil m/k \rceil \times 3n \times 16 \times 2}_{\text{Non-salient weights (2 groups)}} \\
 &\quad + \underbrace{nc + \lceil m/k \rceil \times 2n \times 16 \times 2}_{\text{Salient weights (2 groups)}} \\
 &\quad + \underbrace{n(m+c)}_{\text{Group bitmap}} + \underbrace{m}_{\text{Salient column bitmap}} \\
 &= 2n(m+c) + m + 160n \lceil m/k \rceil.
 \end{aligned} \tag{50}$$

We can derive the BPW equation as

$$\text{BPW}_{\text{HBLLM-row}} = \frac{\mathcal{M}_{\text{HBLLM-row}}}{mn} = 2 + \frac{2nc + m + 160n \lceil m/k \rceil}{mn}. \tag{51}$$

HBLLM-col shares subgroups across two rows and applies intra-band mean sharing, reducing the coefficient overhead:

$$\begin{aligned}
 \mathcal{M}_{\text{HBLLM-col}} &= \underbrace{n(m-c) + \lceil m/k \rceil \times 1.5n \times 16 \times 2}_{\text{Non-salient weights (2 groups)}} \\
 &\quad + \underbrace{nc + \lceil m/k \rceil \times 2n \times 16 \times 2}_{\text{Salient weights (2 groups)}} \\
 &\quad + \underbrace{nm}_{\text{Group bitmap}} + \underbrace{m}_{\text{Salient column bitmap}} \\
 &= 2nm + m + 112n \lceil m/k \rceil.
 \end{aligned} \tag{52}$$

We can derive the BPW equation as

$$\text{BPW}_{\text{HBLLM-col}} = \frac{\mathcal{M}_{\text{HBLLM-col}}}{mn} = 2 + \frac{m + 112n \lceil m/k \rceil}{mn}. \tag{53}$$

F.4. Binary QAT Methods Using Low-Rank Binary Matrices

DBF and LittleBit Double Binary Factorization (DBF) (Boža & Macko, 2026) and LittleBit (Lee et al., 2025a) approximate the weight matrix $\mathbf{W}$ as:

$$\mathbf{W} \approx \widehat{\mathbf{W}} = \text{Diag}(\mathbf{s}_1) (\mathbf{U}_{\pm 1} \text{Diag}(\mathbf{s}_{\text{mid}}) \mathbf{V}_{\pm 1}^{\top}) \text{Diag}(\mathbf{s}_2), \tag{54}$$

where $\mathbf{U}_{\pm 1} \in \{\pm 1\}^{n \times r}$ and $\mathbf{V}_{\pm 1} \in \{\pm 1\}^{m \times r}$ are stored as 1-bit entries, and the scales $\mathbf{s}_1 \in \mathbb{R}^n$, $\mathbf{s}_{\text{mid}} \in \mathbb{R}^r$, $\mathbf{s}_2 \in \mathbb{R}^m$ are stored in FP16. The total storage is:

$$\mathcal{M}_{\text{DBF}} = r(n+m) + 16(n+r+m). \tag{55}$$

We can derive the BPW equation as

$$\text{BPW}_{\text{DBF}} = \frac{r(n+m) + 16(n+r+m)}{mn} \tag{56}$$

E.5. NANOQUANT (Ours)

Our method simplifies the factorization structure by removing the rank-wise scale s_{mid} via a 2-scale system:

$$\mathbf{W} \approx \widehat{\mathbf{W}} = \text{Diag}(\mathbf{s}_1) \mathbf{U}_{\pm 1} \mathbf{V}_{\pm 1}^\top \text{Diag}(\mathbf{s}_2). \quad (57)$$

The total storage required is:

$$\mathcal{M}_{\text{NANOQUANT}} = r(n + m) + 16(n + m). \quad (58)$$

This reduction in scalar overhead contributes to a lower BPW compared to DBF at the same rank r .

We can derive the BPW equation as

$$\text{BPW}_{\text{NANOQUANT}} = \frac{r(n + m) + 16(n + m)}{mn} \quad (59)$$

F.6. Compression Comparison

Compressed Model Comparison. To evaluate the compression capability of each quantization method, we compute the $\text{BPW}_{\text{model}}$ for a model containing L linear layers $\{\mathbf{W}_\ell\}_{\ell=1}^L$ in LLM decoder blocks, with dimensions $n_\ell \times m_\ell$. The total memory bits are given by $\mathcal{M}_{\text{total}} = \sum_{\ell=1}^L \mathcal{M}_\ell$, where $\mathcal{M}_\ell$ is calculated using the formulas of each respective method. The effective bits per weight is:

$$\text{BPW}_{\text{model}} = \frac{\sum_{\ell=1}^L \mathcal{M}_\ell}{\sum_{\ell=1}^L n_\ell m_\ell}. \quad (60)$$

Notably, all open-source implementations of the baseline binary PTQ methods have a maximum value of 50 salient columns ($c \leq 50$) and a unified block size value of $k = 128$. With these constraints, we can derive the theoretical lower and upper bounds of the compression rate of all baselines.

Table 13. Upper and lower bounds of quantized model size (GB) for 1-bit NANOQUANT and various binary post-training quantization baseline methods, represented as (min, max).

Model	BF16	NANOQUANT	BiLLM	STBLLM _{4:8}	STBLLM _{6:8}	STBLLM _{8:8}	ARB-LLM _{RC}	HBLLM _R
L2-7	13.48	1.33	(2.85, 2.86)	(3.36, 3.36)	(3.76, 3.77)	(3.86, 3.87)	(2.55, 2.56)	(3.16, 3.17)
L2-13	26.03	2.24	(5.22, 5.23)	(6.21, 6.22)	(7.00, 7.01)	(7.20, 7.21)	(4.63, 4.64)	(5.81, 5.84)
L2-70	137.95	9.58	(25.65, 25.69)	(31.00, 31.03)	(35.28, 35.30)	(36.35, 36.39)	(22.47, 22.51)	(28.86, 28.94)
L3-1	2.47	0.65	(1.40, 1.40)	(1.48, 1.48)	(1.54, 1.54)	(1.55, 1.56)	(1.36, 1.36)	(1.45, 1.45)
L3-3	6.43	1.14	(2.59, 2.59)	(2.81, 2.81)	(2.99, 2.99)	(3.03, 3.04)	(2.46, 2.47)	(2.72, 2.73)
L3-8	16.06	2.97	(4.61, 4.62)	(5.16, 5.16)	(5.59, 5.60)	(5.70, 5.71)	(4.29, 4.30)	(4.94, 4.96)
L3-70	141.11	12.73	(28.81, 28.85)	(34.15, 34.18)	(38.43, 38.46)	(39.50, 39.54)	(25.62, 25.66)	(32.01, 32.10)
G3-1	2.00	0.69	(1.46, 1.46)	(1.51, 1.52)	(1.56, 1.56)	(1.57, 1.57)	(1.43, 1.43)	(1.49, 1.50)
G3-4	7.76	1.74	(3.84, 3.85)	(4.09, 4.09)	(4.29, 4.29)	(4.34, 4.35)	(3.69, 3.70)	(3.99, 4.00)
G3-12	23.53	3.35	(7.90, 7.91)	(8.74, 8.75)	(9.41, 9.42)	(9.58, 9.59)	(7.40, 7.41)	(8.40, 8.43)
G3-27	54.02	6.00	(14.84, 14.87)	(16.84, 16.86)	(18.44, 18.46)	(18.84, 18.87)	(13.65, 13.68)	(16.04, 16.09)
Q3-0.6	1.19	0.37	(0.78, 0.78)	(0.82, 0.82)	(0.84, 0.84)	(0.85, 0.85)	(0.76, 0.76)	(0.80, 0.81)
Q3-1.7	3.44	0.76	(1.75, 1.76)	(1.86, 1.86)	(1.95, 1.95)	(1.97, 1.98)	(1.69, 1.69)	(1.82, 1.83)
Q3-4	8.04	1.23	(2.86, 2.87)	(3.15, 3.15)	(3.37, 3.38)	(3.43, 3.44)	(2.70, 2.70)	(3.03, 3.05)
Q3-8	16.38	3.35	(4.99, 5.00)	(5.53, 5.53)	(5.96, 5.97)	(6.07, 6.08)	(4.67, 4.68)	(5.31, 5.33)
Q3-14	29.54	4.76	(7.86, 7.87)	(8.89, 8.90)	(9.72, 9.73)	(9.93, 9.94)	(7.25, 7.26)	(8.48, 8.51)

Table 14. Upper and lower bounds of bits-per-weight (BPW) for quantized models of various binary post-training quantization baseline methods, represented as (min, max).

Model	BF16	NANOQUANT	BiLLM	STBLLM _{4:8}	STBLLM _{6:8}	STBLLM _{8:8}	ARB-LLM _{RC}	HBLLM _R
L2-7	16.00	1.00	(2.88, 2.89)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.14)	(2.51, 2.52)	(3.25, 3.27)
L2-13	16.00	1.00	(2.88, 2.88)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.13)	(2.51, 2.51)	(3.25, 3.27)
L2-70	16.00	1.00	(2.88, 2.88)	(3.50, 3.50)	(4.00, 4.00)	(4.13, 4.13)	(2.50, 2.51)	(3.25, 3.26)
L3-1	16.00	1.00	(2.88, 2.90)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.15)	(2.51, 2.53)	(3.25, 3.29)
L3-3	16.00	1.00	(2.88, 2.89)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.14)	(2.51, 2.52)	(3.25, 3.28)
L3-8	16.00	1.00	(2.88, 2.89)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.14)	(2.51, 2.52)	(3.25, 3.27)
L3-70	16.00	1.00	(2.88, 2.88)	(3.50, 3.50)	(4.00, 4.00)	(4.13, 4.13)	(2.50, 2.51)	(3.25, 3.26)
G3-1	16.00	1.00	(2.88, 2.91)	(3.50, 3.52)	(4.00, 4.02)	(4.13, 4.16)	(2.52, 2.55)	(3.25, 3.32)
G3-4	16.00	1.00	(2.88, 2.89)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.14)	(2.51, 2.53)	(3.25, 3.28)
G3-12	16.00	1.00	(2.88, 2.89)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.14)	(2.51, 2.52)	(3.25, 3.27)
G3-27	16.00	1.00	(2.88, 2.88)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.13)	(2.50, 2.51)	(3.25, 3.27)
Q3-0.6	16.00	1.00	(2.88, 2.92)	(3.50, 3.53)	(4.00, 4.03)	(4.13, 4.17)	(2.52, 2.56)	(3.25, 3.33)
Q3-1.7	16.00	1.00	(2.88, 2.90)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.15)	(2.51, 2.53)	(3.25, 3.29)
Q3-4	16.00	1.00	(2.88, 2.89)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.14)	(2.51, 2.52)	(3.25, 3.28)
Q3-8	16.00	1.00	(2.88, 2.89)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.14)	(2.51, 2.52)	(3.25, 3.27)
Q3-14	16.00	1.00	(2.88, 2.88)	(3.50, 3.51)	(4.00, 4.01)	(4.13, 4.13)	(2.51, 2.51)	(3.25, 3.27)

G. LLM Text Generation via Quantized Models

Table 15. Qualitative text generation examples from Llama-2-7B quantized with NANOQUANT at different effective bit-widths. All models are evaluated using the same prompt.

Model	Continuation
Prompt:	The discovery of artificial intelligence is revolutionary, in that
1.00-bit NANOQUANT	it can help solve problems that require a human mind to solve them, or to make a decision. For example, it can identify and predict human patterns, and identify the appropriate actions for those who are in critical situations. It can solve problems that are difficult for humans, such as finding a solution to a puzzle, or solving a problem that requires a human to solve it. It also can help solve problems using human intelligence, such as making a decision to solve the problem, solve a problem or evaluate the situation. In addition, artificial intelligence can help solve problems related to the development of artificial intelligence, such as predicting
0.80-bit NANOQUANT	it has been shown to be able to solve problems normally understood by humans. Surely, the future will be the way that our knowledge is improved, and the development of artificial intelligence will be crucial to future human development. The future is an exciting time for humanity, and our future will be enhanced by the technologies that are being developed. We will be able to achieve the goals that our previous generation have set for humanity. In addition to the information that has been made available in the past, the technology that has been developed is now being used to help us improve our knowledge of humanity, and to expand our
0.55-bit NANOQUANT	it opens the way for many new technological advances. With the creation of computer technology and the development of digital technology, these technologies have resulted in great changes in the world. With the rapid advance of computer technology, computers are now able to operate on a much higher level than previously, and computer technology is now in use in every field of science and technology. Computer technology has also allowed many new technologies and technologies that are now available to both the industrial and the scientific industries. The development of computer technology has also enabled many new innovations in technology and technology that are being developed. These new technologies